The Algebraic Isogeny Model: A General Model with Applications to SQIsign and Key Exchanges

Marius A. Aardal[1], Andrea Basso[2], and Doreen Riepel[3]

[1] Aarhus University, Denmark; maardal@cs.au.dk
[2] IBM Research Europe, Switzerland; andrea.basso@ibm.com
[3] CISPA Helmholtz Center for Information Security, Germany; riepel@cispa.de

Abstract. We introduce the Algebraic Isogeny Model (AIM): an algebraic model, akin to the Algebraic Group Model in the group setting, for isogenies and supersingular elliptic curves. This model is significantly more general than previous ones, such as the Algebraic Group Action Model: the AIM works with arbitrary isogenies over $\mathbb{F}_{p^2}$, rather than being limited to oriented ones, which gives considerably more power to the adversary.
Within this model, we obtain three results. First, we show that any result in the AGAM can be lifted to the AIM, strengthening previous results against more powerful adversaries. Then, we prove that the SQIsign identification protocol is ID-sound: in turn, this implies that SQIsign is EUF-CMA secure in the Quantum Random Oracle Model, resolving (in the AIM) a long-standing open problem. Lastly, we establish the equivalence of the DLOG and CDH problems for all SIDH-derived key exchanges, such as M-SIDH, binSIDH, and terSIDH.

1 Introduction

Isogeny-based cryptography has established itself as a promising solution for post-quantum cryptography, and research in the area is advancing fast. Recently, SQIsign [DKL+20, AAA+25] has advanced to the second round of NIST's Post-Quantum Cryptography Standardization process for additional digital signature schemes. Within a year, several variants and improvements of SQIsign [NOC+24, BDD+24, DF24, DLRW24, SEMR24] have been proposed. At the same time, other paradigms to construct signatures [Ler25, BBC+25, Fou25] are being developed, and the situation is no different for (non-interactive) key exchange and public key encryption (PKE) schemes [FMP23, BF23, BMP23, NO24, BM25, KHK25].

The recent advancements are largely based on higher-dimensional isogenies and complex mathematical algorithms. Consequently, they do not fit within the conceptually simpler and more abstract framework of cryptographic group actions [ADMP20]. The goal is mostly to improve efficiency compared to previous constructions, whereas formal security statements and proofs often get less attention. Security is often argued by explaining known and possibly new attack vectors and why they are expected to fail. Alternatively, new computational problems are introduced, which capture exactly the core of the scheme, therefore being rather ad-hoc. Not least because of SQIsign's success, we can observe that this trend is slowly changing. The broader cryptographic community has developed increased interest in addressing the challenges to improve formal security arguments in isogeny-based cryptography. One obvious target is to capture assumptions more accurately and as "natural" as possible, as for example done in [ABD+25] for SQIsign. As a by-product, this also improves the accessibility to non-experts, be it practitioners or other researchers, as we have seen with cryptographic schemes based on group actions.

One problem arising from this trend is that almost every paper that provides a new construction from isogenies also introduces a new assumption. This is not a new phenomenon in cryptographic research: it is similarly true for certain primitives in classical elliptic curve cryptography, e.g., so-called q-type assumptions [BBG05, Boy08] in the context of (hierarchical) identity-based or attribute-based encryption. However, these scenarios mainly apply to cryptographic constructions with "advanced" functionalities. In isogeny-based cryptography, on the

other hand, we observe this phenomenon even for "simple" and fundamental building blocks like signatures and PKE schemes.[4]

For group-based assumptions, the generic group model (GGM) [Sho97, Mau05] is the main tool to argue about the hardness of new assumptions over elliptic curve groups. Due to structural similarities to classical elliptic curve cryptography, isogeny-based cryptography using group actions was the first target to address the challenges described above. Before going into details about prior models on group actions and how we hope to extend them to the more general case, we first recall some history on models for groups.

Generic and algebraic models. In the GGM, adversaries are given black-box access to the group via an oracle to perform the group operation and equality checks. This allows us to argue about the hardness of problems solely based on the number of queries to those oracles. In practice, the best attacks that we know against problems over elliptic curve groups are indeed generic, and are thus captured by the GGM.

In [FKL18], Fuchsbauer, Kiltz, and Loss introduce the Algebraic Group Model (AGM) which intuitively lies in between the standard model and (a type-safe interpretation of) the GGM. Its main advantage is its simplicity: as opposed to providing information-theoretic lower bounds, it allows relating hardness assumptions and security of schemes via reductions. We observe that the best known classical attacks tend to be not only generic but also algebraic. Here, we informally mean that an adversary only computes new group elements by combining ones it has already seen. In the AGM, this is captured by requiring the adversary to explicitly output this information. For example, if it has received as input the group elements $h_1, \ldots, h_t$ and outputs a group element g, it also needs to explain how g was derived. Specifically, it needs to output a representation $\boldsymbol{c} = (c_1, \ldots, c_t) \in \mathbb{Z}^t$ such that $g = \prod_i h_i^{c_i}$. This additional information can be used by an algebraic reduction and allows us to obtain much stronger security implications compared to the standard model. For example, [FKL18] shows that the discrete logarithm problem is tightly equivalent to the computational Diffie-Hellman problem.

Existing models for isogenies. Since group actions provide a level of abstraction that is easy to work with and very similar to the classical setting, it is only natural to define and use models similar to the AGM and GGM as done in [DHK+23, OZ24], in the following denoted by AGAM and GGAM (algebraic and generic group action model). These models turn out to be intuitively even simpler because there is only the group action operation. At the same time, they even seem to be more realistic than the AGM and GGM for groups. At least classically, it is a long-standing open problem to hash into supersingular isogeny graphs [BBD+22, MMP25], i.e., to sample a supersingular elliptic curve at random without knowing its endomorphism ring. Under this assumption, algorithms are by definition algebraic.

Additional challenges arise if we want to capture quantum algorithms. After all, we are interested in post-quantum security of schemes. Quantum variants of the AGAM and GGAM have been proposed [MZ22, DHK+23, Zha25] and we will discuss some of the subtleties in the main body of this work.

Better guarantees beyond group actions. While useful for group actions, the AGAM does not capture arbitrary isogenies that do not give rise to a group action. These isogenies are much more generic, and are at the basis of a large number of constructions. Concretely, we cannot make statements about schemes like SQIsign, M-SIDH, or POKÉ because they rely on (unoriented) isogenies over $\mathbb{F}_{p^2}$. Yet, these schemes offer unique properties that are not possible in the group-action setting: for instance, a signature scheme as efficient as SQIsign cannot exist from group actions [BGZ23]. At the same time, the security of unoriented constructions is less well understood. For example, we do not have a proof in the quantum random oracle model (QROM) for SQIsign because known lifting results [Unr12, DFMS19, LZ19] do not seem to apply. For

[4] One can even win a prize for "Construct[ing] post-quantum encryption schemes, whose security reduces to (a small variant of) the Endomorphism Ring Problem.", Problem 7, https://isogeni.es/problems.

(variants of) SIDH, the concrete security is often only evaluated in terms of secret key recovery attacks, and not in terms of (session) key recovery or indistinguishability.

Our contributions. In this work, we improve the provable guarantees for isogeny-based schemes in general. We summarize our contributions as follows.

- **A formal model:** We define the Algebraic Isogeny Model (AIM), a general model for isogenies that goes beyond group actions. We provide formal definitions for what we mean with algebraic games and adversaries in this context. We then additionally capture quantum (random) oracles to make statements on security against quantum adversaries.
- **Lifting result:** We establish the relationship between the AIM and the AGAM (with twists) for isogeny-based group actions. We show that the AIM naturally captures the security games captured by the AGAM. This gives evidence for the AIM being a suitable model for capturing all isogeny-based protocols. Furthermore, we show that any reduction in the AGAM can be lifted to the AIM. This strengthens results proven in the AGAM: if a protocol is secure in the AGAM, it is also secure against unrestricted adversaries that use specific properties of isogenies not captured by the group-action framework.
- **Quantum security of SQIsign:** We use our model to prove that the SQIsign identification protocol is ID-sound. The key observation is that in the AIM we can directly extract an isogeny from the explanations. Since we no longer need to rewind the adversary to extract a witness, we obtain a proof of security of SQIsign in the QROM, under the same assumptions used to obtain classical security.
- **Generalized DLOG to CDH equivalence:** We define a generalization of the SIDH key exchange, capturing all variants that have been proposed since the attacks on the original scheme. We then show that the underlying CDH problem and the DLOG problem are equivalent in the AIM, which justifies focusing the cryptanalytic efforts on the secret-key recovery problem.

1.1 Technical overview

Defining the AIM. In an algebraic model for isogenies, we need to properly define what it means for an algorithm to be "algebraic". There are multiple possible interpretations of what this means:

1. An algebraic algorithm produces new curves by computing isogenies from curves it has previously seen.
2. Whenever an algebraic algorithm outputs a curve E, it also "knows" an isogeny from a curve it got as input to E.

The first interpretation matches the intuition behind the AGM (or the AGAM). The only way to compute new elements non-obliviously is to use the group (action). However, for isogenies, this is too restrictive. There are many ways to compute new elliptic curves without directly computing isogenies.

For example, [BBD+22] shows how to hash to curves of known endomorphism ring. Yet, this algorithm is intuitively still algebraic. Given the endomorphism ring of two curves, we can efficiently compute an isogeny between them. Therefore, the hashed curve can always be explained by an isogeny from a starting curve E_{end} of known endomorphism ring. Hence, the second interpretation seems to be the right choice for isogenies.

Building on this intuition, we formalize our model using a typed pseudocode framework inspired by the framework of Jaeger and Mohan [JM24]. There is a type for bits and a type for elliptic curves. The curve type is an abstract type, separate from bits. There are three operations on an elliptic curve type. encode maps a curve to its encoding as bits, and decode goes the other way. The third operation is isogeny, which takes as input a curve E and the representation of an isogeny $\varphi : E \to E'$, and outputs a new curve E'. Additionally, the model fixes a curve E_{end} of known endomorphism ring.

We define what it means for an algorithm $\mathcal{A}$ to be algebraic as follows. Say $\mathcal{A}$ gets as input the curves $E_1, \ldots, E_n$. Then we say that it is algebraic if, whenever it outputs a curve E, it also outputs an isogeny φ such that $\mathsf{isogeny}(E_i, \varphi) = E$ for some $1 \leq i \leq n$ or $\mathsf{isogeny}(E_{\mathsf{end}}, \varphi) = E$. We call the isogeny φ the *explanation* for E. Internally, $\mathcal{A}$ can still use decode and do arbitrary computations on bits, but it must be able to explain its output.

Finally, we generalize our definitions to quantum games. Whenever an algebraic adversary outputs a superposition of curves, it also outputs a superposition of explanations entangled with the curves. This output can then be measured to obtain a classical curve and an explanation.

AGAM to AIM. Isogeny-based group actions are captured by the orientation framework [CK20]. In this framework, the class group $\mathrm{Cl}(\mathfrak{O})$ of a quadratic order $\mathfrak{O}$ acts on the set of $\mathfrak{O}$-oriented supersingular elliptic curves (E, ι).

For our lifting result, we show how to convert between algebraic algorithms in the AGAM with twists ($\mathsf{AGAM}^\top$) and the AIM. Say an algebraic algorithm gets as input the oriented curves $(E_0, \iota_0), \ldots, (E_n, \iota_n)$, and outputs a new curve (E, ι) and an explanation. In the $\mathsf{AGAM}^\top$, the explanation is an ideal class $[\mathfrak{a}] \in \mathrm{Cl}(\mathfrak{O})$ and a bit b for the twist. The explanation satisfies that $[\mathfrak{a}] \star (E_k, \iota_k)$ equals (E, ι) if $b = 0$ and $(E, \bar{\iota})$ if $b = 1$, for some k. In the AIM, the explanation is just an isogeny $\varphi_{\mathsf{expl}} : E_k \to E$, which might not respect the orientations. However, we show that an explanation in the AIM can be efficiently converted to an explanation in the $\mathsf{AGAM}^\top$, and vice versa.

The problem of converting from an $\mathsf{AGAM}^\top$ explanation to an AIM explanation was solved by the Clapoti algorithm [PR23]. The conversion in the other direction proceeds in three steps. In the first step, we show how to find an $\alpha \in \mathfrak{O}$ such that $\psi = \iota(\alpha) \circ \varphi_{\mathsf{expl}} + \varphi_{\mathsf{expl}} \circ \iota_k(\alpha)$ is an oriented isogeny $(E_k, \iota_k) \to (E, \iota)$ or $(E_k, \iota_k) \to (E, \bar{\iota})$. Next, we factor the degree of ψ. This is the only quantum step. Finally, given the factorization, we can recover the corresponding $\mathfrak{O}$-ideal by trial division. For every prime factor ℓ, there are at most two prime $\mathfrak{O}$-ideals of norm ℓ. Using Clapoti, we check which of them divides ψ. Additionally, to handle quantum security games, we also show how to do this conversion in superposition.

Quantum security of SQIsign. $\mathsf{SQIsign}$ is a Fiat-Shamir signature; its underlying Σ-protocol proves knowledge of a non-trivial endomorphism of the public key curve E_{pk}. The protocol starts from a curve E_0 with known endomorphism ring, and then

1. The prover computes a random isogeny $\varphi_{\mathsf{com}} : E_0 \to E_{\mathsf{com}}$ and outputs the curve E_{com} as its commitment,
2. The verifier sends a random challenge isogeny $\varphi_{\mathsf{chl}} : E_{\mathsf{pk}} \to E_{\mathsf{chl}}$,
3. The prover responds with an isogeny $\varphi_{\mathsf{rsp}} : E_{\mathsf{com}} \to E_{\mathsf{chl}}$.

This protocol is special sound: given two transcripts with the same commitment E_{com} but different challenge isogenies φ_{chl} and φ'_{chl}, we can compute a non-scalar endomorphism of E_{pk}. Using the reduction of Page and Wesolowski [PW24], this allows us to efficiently extract the full endomorphism ring of E_{pk}.

The security of $\mathsf{SQIsign}$ in the ROM was formally analyzed in [ABD+25]. They show that in the ROM, $\mathsf{SQIsign}$ is $\mathsf{EUF\text{-}CMA}$-secure assuming the hardness of two problems, $\mathsf{hint\text{-}EndRing}$ and $\mathsf{hint\text{-}dist}$. Here, $\mathsf{hint\text{-}EndRing}$ is the usual endomorphism ring problem, except that the adversary is given some auxiliary input, in the form of several isogenies of large non-smooth degree.

The issue with proving $\mathsf{SQIsign}$ secure in the QROM is extraction: special soundness is not enough to extract a non-scalar endomorphism from the adversary. Classically, after obtaining the first transcript, the extractor can simply rewind the adversary and try again with a new challenge, until it obtains a second transcript. However, this does not work if the adversary is a quantum algorithm, as the measurement of the first transcript might irreversibly collapse the quantum state of the adversary. There are techniques in the literature for overcoming this [Unr12, DFMS19, LZ19], but none of them seem to apply to $\mathsf{SQIsign}$.

Instead, we observe that in the AIM, there is a natural way to extract from quantum adversaries. When the adversary outputs the commitment E_{com}, it also outputs an explanation φ_{expl}. There are two cases, depending on whether $\varphi_{\mathsf{expl}} : E_0 \to E_{\mathsf{com}}$ or $\varphi_{\mathsf{expl}} : E_{\mathsf{pk}} \to E_{\mathsf{com}}$:

1. E_0: since $\mathrm{End}(E_0)$ is known, we know a non-scalar endomorphism α of E_0. Then $\varphi_{\mathsf{expl}} \circ \alpha \circ \hat{\varphi}_{\mathsf{expl}}$ is a non-scalar endomorphism of E_{pk}.
2. E_{pk}: we obtain an endomorphism $\hat{\varphi}_{\mathsf{chl}} \circ \varphi_{\mathsf{rsp}} \circ \varphi_{\mathsf{expl}}$ of E_{pk}. Since φ_{expl} was chosen independently of φ_{chl} and φ_{rsp} is of bounded degree, we can show that with overwhelming probability, the endomorphism is not a scalar.

From this, we show that SQIsign is secure in the AIM+QROM. In Section 5, we prove the security of the version of SQIsign currently submitted to NIST [AAA+25] assuming the hardness of hint-EndRing and hint-dist (the same assumptions used to prove classical security), but the reduction applies to all variants of SQIsign.

Similarly, this technique to extract a non-trivial endomorphism without rewinding also implies that SQIsign is an online-extractable proof of knowledge in the AIM (with respect to the OneEnd language).

SIDH-based key exchanges. After the attacks on SIDH [CD23, MMP+23, Rob23], several key exchanges based on SIDH have been proposed [FMP23, BF23]. The cryptanalytic efforts have focused on solving the key-recovery problem, akin to the DLOG problem in the group setting, rather than CDH-like problem that these key exchanges rely on. However, it is unclear whether the CDH problem is easier than the DLOG one: unlike the CSIDH case [MZ22, GLM24], a CDH-to-DLOG reduction has been a long-standing open problem. In this work, we show that the CDH problem and the DLOG problem for a generalized SIDH-based key exchange are equivalent in the AIM.

While in the group (and group action) setting, showing the equivalence of the DLOG and CDH problems in algebraic models is trivial, the reduction is significantly more complex in our case. All the SIDH-derived key exchanges use a specific set of possible isogenies (e.g., only isogenies of a specific degree), but the adversary in the AIM could output any explanation isogeny, which does not need to be compatible with the key exchange. Moreover, given an arbitrary isogeny between two elliptic curves, we cannot manipulate it to obtain a compatible one.

Our approach, thus, is to guess in advance whether the adversary will produce a compatible isogeny: if it does, we simply pass the DLOG challenge to the CDH oracle and extract a valid DLOG response (i.e., an isogeny compatible with the key exchange) from the adversary's explanation. If we know that the adversary will *not* produce a compatible isogeny, we sample a CDH challenge where we know one of the secret isogenies. When the adversary responds with a non-compatible isogeny, we can extract a non-trivial endomorphism from the composition of the compatible isogeny that we used to generate the CDH challenge and the non-compatible isogeny in the explanation. With this approach, we show that, given access to a CDH oracle, we can solve the DLOG problem or extract a non-trivial endomorphism.

From this, we apply the standard EndRing-to-OneEnd reduction to show that we can solve the DLOG problem or extract the endomorphism ring of a public-key. In other words, assuming that the EndRing problem is hard, DLOG reduces to CDH and thus DLOG and CDH are equivalent (the CDH to DLOG reduction is straightforward). We can go a step further: for all known instantiations of an SIDH-based key exchange, it is possible to extract a compatible isogeny from the knowledge of the endomorphism rings, which implies that DLOG and CDH are equivalent *unconditionally*.

2 Preliminaries

For additional preliminaries on effective group actions (EGAs), quantum computing, Σ-protocols and Fiat–Shamir signatures see Appendix A. For a full description of SQIsign, see [BDD+24, AAA+25].

Notation. We write log for the base-2 logarithm. We let $\mathsf{poly}(x_1, \ldots, x_n)$ denote an unspecified positive polynomial in $x_1, \ldots, x_n$. Similarly, $\mathsf{negl}(x)$ denotes an unspecified negligible function in x. Moreover, we define $\mathrm{polylog}(x_1, \ldots, x_n) = \mathsf{poly}(\log x_1, \ldots, \log x_n)$. We write $\{0,1\}^{\leq n}$ for the set of bit-strings of length at most n. Finally, we write $x \xleftarrow{\$} S$ to denote sampling an element x uniformly at random from a set S.

2.1 Supersingular curves and isogenies

For a prime $p > 3$, let $\mathsf{SS}(p)$ be the set of supersingular elliptic curves defined over $\overline{\mathbb{F}}_p$ up to isomorphism. The isomorphism class of an elliptic curve E is characterized by its j-invariant $j(E)$. Every supersingular j-invariant has a representative curve defined over $\mathbb{F}_{p^2}$. The stationary distribution on $\mathsf{SS}(p)$ is the limit distribution of a random walk in the isogeny graph. We write $E \leftarrow \mathsf{SS}(p)$ when we are sampling a curve from the stationary distribution. A curve $E \in \mathsf{SS}(p)$ is sampled with probability $24/((p-1)|\mathrm{Aut}(E)|)$. The stationary distribution is at statistical distance $O(1/p)$ from the uniform distribution on $\mathsf{SS}(p)$.

There are many approaches for computing isogenies $\varphi : E \to E'$ between curves. They are abstracted by the concept of an *efficient representation*.

Definition 2.1 ([BDD+24]). *Let $\mathbb{F}_q$ be a finite field. An isogeny evaluator $\mathscr{E}$ is a pair of polynomial time algorithms:*

- *$\mathscr{E}.\mathsf{valid}\,(D)$: On input a string $D \in \{0,1\}^*$ it outputs $\perp$ or a triple (E, E', d). In the latter case, E and E' are elliptic curves defined over $\mathbb{F}_q$ and D represents an isogeny $\varphi : E \to E'$ of degree d.*
- *$\mathscr{E}.\mathsf{eval}\,(D, P)$: On input a string $D \in \{0,1\}^*$ and a point $P \in E(\mathbb{F}_{q^k})$, it outputs the image point $\varphi(P) \in E'(\mathbb{F}_{q^k})$ if $\mathscr{E}.\mathsf{valid}\,(D) = (E, E', d)$, otherwise the output is undefined.*

In the case that $\mathscr{E}.\mathsf{valid}\,(D) \neq \perp$ and D is of size polynomial in $\log q$ and $\log d$, we say that D is an efficient representation of φ (with respect to $\mathscr{E}$).

Robert showed in [Rob22, Rob24] that any isogeny φ has an efficient representation using the so-called "8D-representation". The 8D-representation is universal. Given any other efficient representation of φ, we can convert it to the 8D-representation in polynomial time in $\log q$ and $\log d$. The conversion can be computed deterministically, so that every isogeny has a unique 8D-representation.

Definition 2.2. *Given an isogeny $\varphi : E \to E'$, we define its* unique 8D representation *to be the tuple*

$$\big((E, P, Q), (E', \varphi(P), \varphi(Q)), \deg \varphi\big),$$

where P, Q is a deterministic basis of $E[N]$ and the value N is the smallest product of primes $N = \prod_{q_i \neq p} q_i$ such that $N > \sqrt{\deg \varphi}$.

We can compute several operations on efficient representations.

Lemma 2.3 ([Rob24]). *For each of the operations below there is a polynomial time algorithm which operates on efficient representations:*

- ***Dual:** On input $\varphi : E_1 \to E_2$, compute $\widehat{\varphi} : E_2 \to E_1$.*
- ***Equality check:** On input $\varphi, \psi : E_1 \to E_2$, check whether $\varphi = \psi$.*
- ***Composition:** On input $\varphi : E_1 \to E_2$ and $\psi : E_2 \to E_3$, compute $\psi \circ \varphi : E_1 \to E_3$.*
- ***Splitting:** On input $\varphi : E_1 \to E_2$ and coprime n_1, n_2 s.t. $\deg(\varphi) = n_1 n_2$, find φ_1, φ_1' of degree n_1 and φ_2, φ_2' of degree n_2 s.t. $\varphi = \varphi_2' \circ \varphi_1 = \varphi_1' \circ \varphi_2$.*
- ***Division:** On input $\varphi : E_1 \to E_2$, $\varphi_l : E_1 \to E_2'$ and $\varphi_r : E_1' \to E_2$, check if there exist φ_l', φ_r' s.t. $\varphi = \varphi_r' \circ \varphi_l$ or $\varphi = \varphi_r \circ \varphi_l'$, and if so output them.*
- ***Pushforward:** On input $\varphi : E_1 \to E_2$ and $\psi : E_1 \to E_3$ with coprime degrees, compute a pushforward $\varphi' : E_3 \to E_{23}$ of φ by ψ.*

2.2 Isogeny-based group actions

Isogeny-based group actions, like CSIDH [CLM+18], are captured by the orientation framework [CK20, Onu21, Wes22]. In the following, K is a quadratic imaginary field and $\mathfrak{O}$ is an order in K.

Definition 2.4 (Orientation). *A K-orientation on an elliptic curve E is an embedding $\iota : K \hookrightarrow \mathrm{End}(E) \otimes \mathbb{Q}$. A K-orientation on E is an $\mathfrak{O}$-orientation if $\iota(K) \cap \mathrm{End}(E) = \iota(\mathfrak{O})$. In this case, we say that (E, ι) is an $\mathfrak{O}$-oriented curve.*

Let (E, ι) be a K-oriented curve, and let $\alpha \in K$ such that $\iota(\alpha) \in \mathrm{End}(E)$. Then $\iota(\overline{\alpha}) = \widehat{\iota(\alpha)}$ and $\deg \iota(\alpha) = \mathrm{N}(\alpha)$. An isogeny $\varphi : E \to E'$ induces a K-orientation $\varphi_*(\iota)$ on E', $\iota_*(\iota)(\beta) = (\varphi \circ \iota(\beta) \circ \widehat{\varphi}) \otimes \frac{1}{\deg \varphi}$.

Definition 2.5 (Oriented isogeny). *Given two K-oriented curves (E, ι_E) and (F, ι_F), an isogeny $\varphi : E \to F$ is K-oriented if $\iota_F = \varphi_*(\iota_E)$. We denote this by $\varphi : (E, \iota_E) \to (F, \iota_F)$. If $\deg \varphi = \ell$ is prime, ι_E is an $\mathfrak{O}$-orientation and ι_F is an $\mathfrak{O}'$ orientation, then the isogeny is one of three types:*

- *Horizontal: If $\mathfrak{O} = \mathfrak{O}'$.*
- *Ascending: If $\mathfrak{O} \subset \mathfrak{O}'$ with relative conductor ℓ.*
- *Descending: If $\mathfrak{O}' \subset \mathfrak{O}$ with relative conductor ℓ.*

If $\deg \varphi$ is composite, we say that it is horizontal, ascending or descending if it factors into prime degree isogenies all of that type.

We let $\mathsf{SS}_{\mathfrak{O}}(p)$ be the set of $\mathfrak{O}$-oriented supersingular curves over $\overline{\mathbb{F}_p}$ up to K-oriented isomorphism. Abusing notation, we will often write $(E, \iota) \in \mathsf{SS}_{\mathfrak{O}}(p)$ for a representative of the isomorphism class (over $\mathbb{F}_p$ or $\mathbb{F}_{p^2}$). For a prime $\ell \neq p$, the oriented ℓ-isogeny graph forms a volcano structure.

Theorem 2.6 ([CK20]). *Let $(E, \iota) \in \mathsf{SS}_{\mathfrak{O}}(p)$, $\ell \neq p$ be a prime and let $L = \left(\frac{\mathrm{disc}\,\mathfrak{O}}{\ell}\right)$ be the Legendre symbol. Then we can classify the degree ℓ isogenies from (E, ι) as follows (up to post-composition by K-oriented isomorphisms):*

- *There are $\ell - L$ descending isogenies.*
- *If ℓ divides the conductor of $\mathfrak{O}$, there is one ascending isogeny and no horizontal isogenies.*
- *If ℓ does not divide the conductor of $\mathfrak{O}$, there are no ascending isogenies and $L+1$ horizontal isogenies.*

For $\ell = p$, there is only one isogeny, the p-power Frobenius map π, and it is horizontal.

An $\mathfrak{O}$-ideal $\mathfrak{a}$ induces an isogeny $\varphi_{\mathfrak{a}} : E \to E_{\mathfrak{a}}$ of kernel $E[\mathfrak{a}]$ and degree $\mathrm{N}(\mathfrak{a})$. We obtain an action of invertible $\mathfrak{O}$-ideals on $\mathsf{SS}_{\mathfrak{O}}(p)$ given by $\mathfrak{a} \diamond (E, \iota) = (E_{\mathfrak{a}}, (\varphi_{\mathfrak{a}})_*(\iota))$. The twisting involution $\tau : \mathsf{SS}_{\mathfrak{O}}(p) \to \mathsf{SS}_{\mathfrak{O}}(p)$ is defined as $\tau(E, \iota) = (E, \overline{\iota})$, where $\overline{\iota}(\alpha) = \iota(\overline{\alpha})$. It satisfies $\tau(\mathfrak{a} \diamond (E, \iota)) = \overline{a} \diamond \tau(E, \iota)$.

Theorem 2.7. *The action*

$$\star : \mathrm{Cl}(\mathfrak{O}) \times \mathsf{SS}_{\mathfrak{O}}(p) \to \mathsf{SS}_{\mathfrak{O}}(p), ([\mathfrak{a}], (E, \iota)) \mapsto \mathfrak{a} \diamond (E, \iota)$$

is free and has at most two orbits. There is exactly one non-empty orbit if and only if p is ramified in K and does not divide the conductor of $\mathfrak{O}$.

The theorem follows from [Onu21, Proposition 3.3], [Onu21, Theorem 3.4] and [ACL+22, Theorem 4.4].

Definition 2.8 (Representations of orientations). *Let $(E, \iota) \in \mathsf{SS}_{\mathfrak{O}}(p)$. A representation D of the $\mathfrak{O}$-orientation ι is an encoding of a generator ω of $\mathfrak{O}$, i.e. $\mathfrak{O} = \mathbb{Z}[\omega]$, together with a representation of the endomorphism $\iota(\omega)$. We say that D is an efficient representation when it is of size polynomial in $\log |\operatorname{disc}(\mathfrak{O})|$.*

2.3 Quantum computing

Qubit. A qubit A is a quantum system associated to a two-dimensional complex Hilbert space $\mathscr{H}_A$. $|\psi\rangle \in \mathscr{H}_A$ is a column vector, and $\langle\psi|$ its conjugate transpose. The pure state of the qubit A is described by a vector $|\psi\rangle \in \mathscr{H}_A$ of norm $\||\psi\rangle\| = \sqrt{\langle\psi|\psi\rangle} = 1$. In the computational basis, we have $|\psi\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle$ with $\alpha_0, \alpha_1 \in \mathbb{C}$ and $|\alpha_0|^2 + |\alpha_1|^2 = 1$.

Multi-qubit registers. The joint state of two qubits A and B is defined over $\mathscr{H}_A \otimes \mathscr{H}_B$. The product state of $|\psi_A\rangle \in \mathscr{H}_A$ and $|\psi_B\rangle \in \mathscr{H}_B$ is $|\psi_A\rangle |\psi_B\rangle \coloneqq |\psi_A\rangle \otimes |\psi_B\rangle$. We write $|\psi\rangle_A$ to denote that the state is for the register A.

The pure state of an n-qubit register is a unit vector $|\psi\rangle = \sum_{x\in\{0,1\}^n} \alpha_x |x\rangle$. It is said to be in superposition if at least two coefficients α_x and α_y are non-zero. It is said to be entangled if it cannot be written as the tensor of one-qubit states.

Operations. There are two types of operations on qubits, unitary transformations and measurements. If an n-qubit state is measured in the computational basis, the state collapses to $|x\rangle$ with probability $|\alpha_x|^2$. By the deferred measurement principle, any measurement can be delayed to the end of the computation, without changing the probability distribution of the result.

2.4 Quantum algorithms and oracles

Oracles. We consider two kinds of oracles, quantum oracles and classical oracles. A quantum oracle O is a unitary transformation on three registers: An input register X_{O}, an output register Y_{O} and a private state register S_{O}. O first computes the output with the unitary U_Y^{O}. On the computational basis, it is the map

$$U_Y^{\mathsf{O}} : |x\rangle |y\rangle |\mathsf{state}\rangle \mapsto |x\rangle |y \oplus f_{\mathsf{O}}(x, \mathsf{state})\rangle |\mathsf{state}\rangle$$

where f_{O} is some function. Then it computes the new state with a unitary

$$U_S^{\mathsf{O}} : |x\rangle |y\rangle |\mathsf{state}\rangle \mapsto |x\rangle |y\rangle |s_{\mathsf{O}}(x, \mathsf{state})\rangle .$$

We emphasize that the state update must be reversible. We define a classical oracle in the same way, except that the operation U_S^{O} does not have to be unitary, and that it initially measures all of its registers.

The quantum random oracle RO is an oracle implementing a random function $f : \{0,1\}^{\leq u} \to \mathcal{C}$. Its state is initialized to the uniform superposition of function tables for f.

Quantum oracle algorithms. A quantum oracle algorithm $\mathcal{A}$ is given access to oracles $\mathsf{O}_1, \ldots, \mathsf{O}_m$. It maintains an input register $X_{\mathcal{A}}$, an output register $Y_{\mathcal{A}}$ and a state register $S_{\mathcal{A}}$. The algorithm can also access the input and output registers of the oracles, but not their state registers.

We define $\mathcal{A}$ by a sequence of unitaries $U_1, \ldots, U_t$ and indices $q_1, \ldots, q_{t-1} \in \{1, \ldots, m\}$. Initially, the register $X_{\mathcal{A}}$ is prepared with the input of $\mathcal{A}$. Then we run the sequence of operations $U_1, \mathsf{O}_{q_1}, \ldots, U_{t-1}, \mathsf{O}_{q_{t-1}}, U_t$. The unitary U_i prepares the query for the oracle O_{q_i} in the oracle's input register. After computing U_t, the output of $\mathcal{A}$ is in $Y_{\mathcal{A}}$. If the output is supposed to be classical, we measure it.

2.5 Σ-protocols

A binary NP relation R is a set of tuples $(x, w) \in \mathcal{X} \times \mathcal{W} \subseteq \{0,1\}^* \times \{0,1\}^*$. We refer to x as the *statement* and w as the *witness*. Given (x, w), we can check if it is in R in polynomial time in $|x|$.

Here, a PPT algorithm is a probabilistic polynomial time algorithm in the size of the public parameters pp. An instance generator Gen_R for R is a PPT algorithm that outputs a pair (x, w) such that $(x, w) \in R$. All relations that we consider come equipped with an instance generator.

Game $\mathsf{wHVZK}_{\Sigma,b}(\mathcal{A})$	**Procedure** O_0	**Procedure** O_1
00 $(x,w) \leftarrow \mathsf{Gen}_R$	04 $(\mathsf{com},\mathsf{state}) \leftarrow \mathcal{P}_1(x,w)$	08 $(\mathsf{com},\mathsf{chl},\mathsf{rsp}) \leftarrow \mathcal{S}(x)$
01 $(\mathsf{com},\mathsf{chl},\mathsf{rsp}) \leftarrow \mathsf{O}_b$	05 $\mathsf{chl} \xleftarrow{\$} \mathcal{C}$	09 **return** $(\mathsf{com},\mathsf{chl},\mathsf{rsp})$
02 $b' \leftarrow \mathcal{A}(x,\mathsf{com},\mathsf{chl},\mathsf{rsp})$	06 $\mathsf{rsp} \leftarrow \mathcal{P}_2(\mathsf{state},\mathsf{chl})$	
03 **return** b'	07 **return** $(\mathsf{com},\mathsf{chl},\mathsf{rsp})$	

Fig. 1: wHVZK game for Σ with simulator $\mathcal{S}$. The game is defined relative to a bit $b \in \{0,1\}$.

Definition 2.9 (Hard relation). *Let R be a relation. It has an associated game, where the adversary $\mathcal{A}$ is given a statement and has to find an associated witness. The advantage of $\mathcal{A}$ in this game is*

$$\mathsf{Adv}^{\mathsf{rel}}_R(\mathcal{A}) := \Pr\left[(x,w^*) \in R \;\middle|\; (x,w) \leftarrow \mathsf{Gen}_R(1^\lambda),\ w^* \leftarrow \mathcal{A}(x)\right].$$

Definition 2.10 (Σ-protocol). *A Σ-protocol for a relation R is a 3-message interactive protocol between a prover and a verifier, given by a tuple of PPT algorithms $(\mathcal{P}_1,\mathcal{P}_2,\mathsf{Ver}_\Sigma)$ as follows:*

1. *On input $(x,w) \in R$, the prover runs $(\mathsf{com},\mathsf{state}) \leftarrow \mathcal{P}_1(x,w)$, obtaining a commitment com and the state state needed for the response computation. It sends com to the verifier.*
2. *The verifier samples a challenge chl uniformly from the challenge space $\mathcal{C}$ and sends it to the prover.*
3. *The prover replies to the verifier with the response $\mathsf{rsp} \leftarrow \mathcal{P}_2(\mathsf{state},\mathsf{chl})$.*
4. *Finally, the verifier runs the verification algorithm to obtain a bit $b \leftarrow \mathsf{Ver}_\Sigma(x,\mathsf{com},\mathsf{chl},\mathsf{rsp})$. It accepts if and only if $b = 1$.*

The Σ-protocol should always have *correctness*. If the prover and verifier run the protocol honestly, the verifier should accept except with negligible probability in λ. In this paper, we are also interested in two other properties.

Definition 2.11 (wHVZK). *Assume that Σ comes with a PPT algorithm $\mathcal{S}$, called the simulator. We define weak-honest-verifier zero-knowledge (*wHVZK*) by the game in Fig. 1. The advantage of an adversary $\mathcal{A}$ playing the game is*

$$\mathsf{Adv}^{\mathsf{wHVZK}}_\Sigma(\mathcal{A}) := \left|\Pr\left[\mathsf{wHVZK}_{\Sigma,0}(\mathcal{A}) \to 1\right] - \Pr\left[\mathsf{wHVZK}_{\Sigma,1}(\mathcal{A}) \to 1\right]\right|.$$

Definition 2.12 (Commitment min-entropy). *We define*

$$\mathsf{MinEnt}(\Sigma) := \max_{(x,w)} \max_{\mathsf{com}} \Pr\left[\mathsf{com} = \mathsf{com}' \;\middle|\; (\mathsf{com}',\mathsf{state}) \leftarrow \mathcal{P}_1(x,w)\right],$$

where the first max ranges over the pairs that might be output by Gen_R. The commitment min-entropy of Σ is $-\log(\mathsf{MinEnt}(\Sigma))$.

2.6 Signature schemes in the QROM

We consider the quantum random oracle model (QROM), where all algorithms are given black-box access to the quantum oracle RO, implementing a uniformly random function $\{0,1\}^{\leq u} \to \mathcal{C}$. For $x \in \{0,1\}^{\leq u}$, it implements the map

$$|x\rangle_X |y\rangle_Y \mapsto |x\rangle_X |x \oplus \mathsf{RO}(x)\rangle_Y.$$

Algorithms can query the random oracle on a superposition of inputs. We usually omit the dependency on the random oracle from our notation, writing $\mathcal{A}$ instead of $\mathcal{A}^{\mathsf{RO}}$.

Definition 2.13 (Signature scheme). *Let M be some set. A signature scheme Sig for a message space M is a tuple of PPT algorithms $(\mathsf{Gen},\mathsf{Sign},\mathsf{Ver})$ defined as follows.*

Game $\mathsf{EUF\text{-}CMA}_{\mathsf{Sig}}(\mathcal{A})$	**Oracle** $\mathsf{O}_{\mathrm{SIGN}}(\mathsf{msg})$
00 $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Gen}$	05 $\mathsf{sig} \leftarrow \mathsf{Sign}(\mathsf{sk}, \mathsf{msg})$
01 $\mathcal{Q} := \emptyset$	06 $\mathcal{Q} := \mathcal{Q} \cup \{\mathsf{msg}\}$
02 $(\mathsf{msg}^\star, \mathsf{sig}^\star) \leftarrow \mathcal{A}^{\mathsf{O}_{\mathrm{SIGN}}}(\mathsf{pk})$	07 **return** sig
03 **if** $\mathsf{msg}^\star \in \mathcal{Q}$ **return** 0	
04 **return** $\mathsf{Ver}(\mathsf{pk}, \mathsf{msg}^\star, \mathsf{sig}^\star)$	

Fig. 2: The EUF-CMA-game for a signature scheme Sig. EUF-NMA is defined similarly, but does not give access to oracle $\mathsf{O}_{\mathrm{SIGN}}$.

- Gen*: The key generation algorithm outputs the public key* pk *and the secret key* sk*.*
- Sign(sk, msg)*: On input the secret key* sk *and a message* $\mathsf{msg} \in M$*, the signing algorithm outputs a signature* sig*.*
- Ver(pk, msg, sig)*: On input the public key* pk*, a message* msg *and a signature* sig*, the verification algorithm outputs 1 (accept) or 0 (reject).*

A signature scheme should always have correctness. In other words, the verification of a correctly generated signature should only fail with negligible probability. In addition, we require EUF-CMA security.

Definition 2.14 (EUF-CMA/NMA security). *For a signature scheme* Sig*, we define existential unforgeability under chosen message attacks* (EUF-CMA) *via the game shown in Fig. 2. The advantage of an adversary* $\mathcal{A}$ *playing the game is*

$$\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{Sig}}(\mathcal{A}) := \Pr[\mathsf{EUF\text{-}CMA}_{\mathsf{Sig}}(\mathcal{A}) = 1].$$

Existential unforgeability under no-message attacks (EUF-NMA) *is a special case, where the adversary cannot query the signing oracle, and it is defined analogously.*

Σ-protocols can be used to construct signature schemes in the QROM, using the *Fiat–Shamir transform*. The Fiat–Shamir transform $\mathsf{FS}[\Sigma]$ of Σ is the non-interactive protocol where the challenge is obtained as $\mathsf{chl} \leftarrow \mathsf{RO}(\mathsf{com}, x)$.

Definition 2.15 (Fiat–Shamir signature). *Let* Σ *be a* Σ*-protocol for a relation* R *and let* $M \subseteq \{0,1\}^*$*. We define the signature scheme* $\mathsf{SIG}[\Sigma]$ *as follows.*

- Gen*: Sample* $(x, w) \leftarrow \mathsf{Gen}_R$*. Set* $\mathsf{pk} := x$ *and* $\mathsf{sk} := (x, w)$*, and output* $(\mathsf{pk}, \mathsf{sk})$*.*
- Sign(sk, msg)*: Generate a transcript* $(\mathsf{com}, \mathsf{chl}, \mathsf{rsp})$ *using the prover algorithm in* $\mathsf{FS}[\Sigma]$*, but compute the challenge as* $\mathsf{chl} \leftarrow \mathsf{RO}(\mathsf{com}, x, \mathsf{msg})$*. Output* $\mathsf{sig} := (\mathsf{com}, \mathsf{chl}, \mathsf{rsp})$*.*
- Ver(pk, msg, sig)*: Write* $\mathsf{sig} = (\mathsf{com}, \mathsf{chl}, \mathsf{rsp})$*. It outputs 1 if and only if* $\mathsf{Ver}_\Sigma(x, \mathsf{com}, \mathsf{chl}, \mathsf{rsp}) = 1$ *and* $\mathsf{chl} = \mathsf{RO}(\mathsf{com}, x, \mathsf{msg})$*.*

Observe that $\mathsf{SIG}[\Sigma]$ is essentially the Fiat–Shamir transform of Σ, but with the relation modified to $R^* = \{(x, \mathsf{msg}, w) \mid (x, w) \in R, \mathsf{msg} \in M\}$.

3 The Algebraic Isogeny Model

For the sake of presentation, we begin by defining our model for classical games. Later, we extend our definitions to also incorporate quantum oracles.

Typed pseudocode framework. Our model is written in typed pseudocode, inspired by the framework of Jaeger and Mohan [JM24]. There are two types: $\langle bits \rangle$ and $\langle curve \rangle$ (later, we will introduce their quantum equivalents). The curve type $\langle curve \rangle$ represents a curve $E \in \mathsf{SS}(p)$, i.e. a supersingular elliptic curve E up to isomorphism. We view a $\langle curve \rangle$ object as an abstract object, separate from bits. However, a $\langle curve \rangle$ object has a unique encoding into bits, given by its j-invariant $j(E)$.

In the framework, any operation on bits is allowed, whereas there are only three operations permitted on the curve type:

- $\mathsf{encode} : \langle curve \rangle \to \langle bits \rangle$. The encoding operation outputs the j-invariant of the curve object.
- $\mathsf{decode} : \langle bits \rangle \to \langle curve \rangle \cup \{\bot\}$. The decoding operation maps a j-invariant to its associated curve object. It outputs $\bot$ if the input is not valid.
- $\mathsf{isogeny}_{\mathsf{rep}} : \langle curve \rangle \times \langle bits \rangle \to (\langle curve \rangle \cup \{\bot\}) \times \langle bits \rangle$. The isogeny operation takes as input an elliptic curve E and an efficient representation of an isogeny $\varphi : E \to E'$, and outputs the curve E' (and potentially some extra bits). Here, rep refers to the type of efficient representation used for the isogeny. The operation outputs $\bot$ if the input is not valid.

In general, there are many types of efficient representations of isogenies. Each type has its associated $\mathsf{isogeny}$ operation. When rep is clear from context, we usually drop the subscript and just write $\mathsf{isogeny}$. Typically, the representation does not produce any auxiliary bits. In such cases, we omit this component from the notation.

We view other elliptic curve objects like torsion points simply as bit-strings. The adversary can evaluate points or perform arbitrary operations on those bit-strings, e.g., to try to compute new curves with $\mathsf{isogeny}$. In practice, we will not write out the type of every object. To make curve objects stand out, we will use boldface letters (e.g. $\mathbf{E}$).

Public parameters. Security games are always specified relative to some public parameters pp. These parameters always include:

- p: the prime characteristic over which the supersingular curves are defined.
- $\mathbf{E}_{\mathsf{end}}$: A supersingular curve of known endomorphism ring.
- $1, \beta_1, \beta_2, \beta_3$: A basis of $\mathrm{End}(\mathbf{E}_{\mathsf{end}})$ as a $\mathbb{Z}$-lattice. Each endomorphism is in the unique 8D-representation and of degree polynomial in $\log p$.

All our games are parameterized by pp, so we omit it from the notation.

Algebraic games. A single-stage security game G is specified by two procedures $\mathsf{Initialize}$ and $\mathsf{Finalize}$ and a list of oracles $\mathsf{O}_1, \ldots \mathsf{O}_n$.

- Game state: There is a game state S shared by all the procedures and oracles.
- $\mathsf{Initialize}$: Runs at the beginning of the game to generate the input for the adversary and to set up the game state S.
- $\mathsf{Finalize}$: Runs at the end of the game on the adversary's output and the game state S. It outputs 1 if the adversary has won the game, else it outputs 0.

We define the advantage of an adversary $\mathcal{A}$ in the game G as the probability that it wins the game, $\mathsf{Adv}^{\mathsf{G}}(\mathcal{A}) \coloneqq \Pr\left[\mathsf{G}(\mathcal{A}) = 1\right]$. The running time of $\mathcal{A}$ in G is denoted $\mathsf{Time}^{\mathsf{G}}(\mathcal{A})$. Both are defined with respect to the public parameters pp.

We say that G is an *algebraic game* if the game does not use decode. An algebraic game only computes new curves using the $\mathsf{isogeny}$ operation. An example of an algebraic game is presented on the left of Fig. 3.

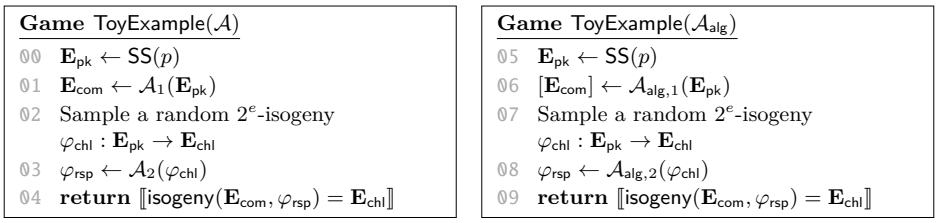

Fig. 3: **Left:** A toy example of an algebraic game for Σ-protocols like SQIsign, with an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$. For simplicity, we present it as a two-stage game, where $\mathcal{A}_1$ and $\mathcal{A}_2$ share a state. Elliptic curves are written in bold letters. **Right:** An algebraic adversary $\mathcal{A}_{\mathsf{alg}}$ playing the algebraic game. The algebraic adversary outputs $[\mathbf{E}_{\mathsf{com}}] = (\mathbf{E}_{\mathsf{com}}, \varphi_{\mathsf{expl}})$, where the explanation φ_{expl} is an isogeny $\mathbf{E}_{\mathsf{end}} \to \mathbf{E}_{\mathsf{com}}$ or $\mathbf{E}_{\mathsf{pk}} \to \mathbf{E}_{\mathsf{com}}$.

Algebraic algorithms. Let $\mathcal{A}$ be an algorithm that takes as input the curves $\mathbf{E}_1, \ldots, \mathbf{E}_n$ and possibly some bits. We say that $\mathcal{A}$ is algebraic if, whenever it outputs a curve $\mathbf{E}$, it also outputs a valid isogeny φ_{expl} of the form $\mathbf{E}_{\mathsf{end}} \to \mathbf{E}$ or $\mathbf{E}_i \to \mathbf{E}$ for some $i \in \{1, \ldots, n\}$. We call φ_{expl} an *explanation* for $\mathbf{E}$. For simplicity, we require that φ_{expl} is given in the unique 8D-representation. We write $[\mathbf{E}]$ for the joint output $(\mathbf{E}, \varphi_{\mathsf{expl}})$. See the right side of Fig. 3 for an example.

We emphasize that an algebraic algorithm is allowed to use decode. However, when it outputs a curve $\mathbf{E}$, it needs to explain the curve with an isogeny φ_{expl}.

When an algebraic adversary $\mathcal{A}$ is playing an algebraic game G, the game is not allowed to depend on the explanations. This means that when the algebraic adversary outputs an explanation, it is simply ignored. The explanations are only used for reductions, which we will discuss next.

Algebraic reductions. Let G and H be algebraic games. A reduction R from H to G is an algorithm such that for every adversary G playing G, $\mathcal{B} := \mathsf{R}^{\mathcal{A}}$ is an adversary for H. We say that R is an *algebraic reduction* if, whenever $\mathcal{A}_{\mathsf{alg}}$ is an algebraic adversary for G, $\mathcal{B}_{\mathsf{alg}} = \mathsf{R}^{\mathcal{A}_{\mathsf{alg}}}$ is an algebraic adversary for H. Note that when $\mathcal{B}_{\mathsf{alg}}$ runs $\mathcal{A}_{\mathsf{alg}}$ internally, it can use the explanations output by $\mathcal{A}_{\mathsf{alg}}$.

3.1 Discussion

In this section, we discuss some of the design choices made in the definition of the AIM. We extend this discussion in Appendix B.

The curve type. In general, there are many different ways to represent curves. There are many different curve models, like the Weierstrass, Montgomery, and Edwards curves. Sometimes we work with a fixed curve, and other times we are only working with curves up to isomorphism. We want a concrete definition that encompasses all of these cases.

The key observation is that we can always represent a supersingular isomorphism class by a canonical representative $\mathbf{E}$ defined over $\mathbb{F}_{p^2}$. With this interpretation, $\mathbf{E}$ is a fixed curve, and notations like $\mathbf{E}[2^e]$ are well defined.

Alternatively, if we want to pick another representative $E' \cong \mathbf{E}$, this can be specified with some extra bits. We can efficiently compute an isomorphism $\alpha : E \to E'$ in the 8D-representation. Then E' can be represented as a pair $(\mathbf{E}, \alpha) \in \langle curve \rangle \times \langle bits \rangle$.

The stationary distribution. Curiously enough, there is no known algebraic algorithm for sampling perfectly from the stationary (or even the uniform) distribution on $\mathsf{SS}(p)$. The stationary distribution is the limiting distribution of the random walk. Thus, while the walk converges rapidly, it only approaches the stationary distribution asymptotically. Accordingly, the notation $\mathbf{E} \leftarrow \mathsf{SS}(p)$ should be interpreted as algebraic sampling that is statistically close to the stationary distribution. Since the resulting statistical distance is negligible in $\log p$, we ignore this distinction for simplicity.

The standard model and the ROM. We can express all standard model games in our typed pseudocode framework. While the standard model is not defined in typed pseudocode, protocols are usually defined such that it is clear which variables are curves and which are not. Hence, there is usually a natural interpretation of standard model games in our typed pseudocode.

The ROM can be modeled by adding a classical oracle RO implementing a random function. We emphasize that if an algebraic adversary outputs a curve to RO, it also outputs an explanation. This intuition extends to the QROM.

3.2 Extension to quantum games

In this section, we extend our model to quantum games. We do not cover all quantum games[5]: our focus on the QROM and related "post-quantum" games.

Typed pseudocode framework. We extend the model to (typed) quantum pseudocode. By quantum pseudocode, we mean that variables are registers that can be in superposition. We introduce two new types, $\langle qbits \rangle$ and $\langle qcurve \rangle$, for quantum registers holding bits and curves respectively. The types $\langle bits \rangle$ and $\langle curve \rangle$ are now used to distinguish for classical registers.

We write $\mathtt{Q}.R$ to denote a quantum bit register labeled R and $\mathtt{Q}.\mathbf{E}$ to denote a quantum curve register labeled $\mathbf{E}$. A classical curve is simply written as $\mathbf{E}$.

Any unitary transformation or measurement is allowed on $\langle qbits \rangle$ registers. For $\langle qcurve \rangle$ registers, the operations $\mathsf{encode}, \mathsf{decode}$ and $\mathsf{isogeny}$ can now be computed in superposition. Some care is needed to define them properly, so that the operations are reversible:

- $\mathtt{encode} : \langle qcurve \rangle \to \langle qbits \rangle$. When run in superposition, we write $\mathtt{Q}.O \leftarrow \mathtt{encode}(\mathtt{Q}.\mathbf{E})$ to denote the following: encode is run on an input register $\mathtt{Q}.\mathbf{E}$ and output register $\mathtt{Q}.O$ via the operation $|x\rangle_{\mathbf{E}} |y\rangle_O \mapsto |x\rangle_{\mathbf{E}} |y \oplus \mathsf{encode}(\mathbf{E})\rangle_O$.
- $\mathtt{decode} : \langle qbits \rangle \to \langle qcurve \rangle \cup \{\bot\}$. We write $\mathtt{Q}.\mathbf{E} \leftarrow \mathtt{decode}(\mathtt{Q}.I)$ to denote running the decoding operation on an input register $\mathtt{Q}.I$ and an output register $\mathtt{Q}.\mathbf{E}$. Since the output is a quantum curve type, we want to avoid the operation $\oplus$ on the output register. Instead, $\mathtt{decode}$ internally first clears the output register via measuring and then computes $|y\rangle_I |\bot\rangle_{\mathbf{E}} \mapsto |y\rangle_I |\mathsf{decode}(y)\rangle_{\mathbf{E}}$.
- $\mathtt{isogeny}_{\mathsf{rep}} : \langle qcurve \rangle \times \langle qbits \rangle \to (\langle qcurve \rangle \cup \{\bot\}) \times \langle qbits \rangle$. We write $(\mathtt{Q}.\mathbf{E}', \mathtt{Q}.\mathsf{aux}) \leftarrow \mathtt{isogeny}_{\mathsf{rep}}(\mathtt{Q}.\mathbf{E}, \mathtt{Q}.X)$ to denote running the isogeny operation on two input registers $\mathtt{Q}.\mathbf{E}$, $\mathtt{Q}.X$ and two output registers $\mathtt{Q}.\mathbf{E}'$, $\mathtt{Q}.\mathsf{aux}$, where the latter captures potential auxiliary qubits. Similarly to $\mathtt{decode}$, it first clears the output register to then compute $|\mathbf{F}\rangle_{\mathbf{E}} |\varphi\rangle_X |\bot\rangle_{\mathbf{E}',\mathsf{aux}} \mapsto |\mathbf{F}\rangle_{\mathbf{E}} |\varphi\rangle_X |\mathsf{isogeny}_{\mathsf{rep}}(\mathbf{F}, \varphi)\rangle_{\mathbf{E}',\mathsf{aux}}$.

When we write that the operation internally clears the register via measuring, this should only be interpreted as enforcing that a fresh register is used, which is without loss of generality. In addition, we need four additional operations:

- $\mathsf{swap} : \langle qcurve \rangle \times \langle qcurve \rangle \times \langle qbit \rangle \to \langle qcurve \rangle$. A controlled swap gate, mapping $|\mathbf{E}\rangle |\mathbf{E}'\rangle |0\rangle \mapsto |\mathbf{E}\rangle |\mathbf{E}'\rangle |0\rangle$ and $|\mathbf{E}\rangle |\mathbf{E}'\rangle |1\rangle \mapsto |\mathbf{E}'\rangle |\mathbf{E}\rangle |1\rangle$.
- $\mathsf{measure} : \langle qcurve \rangle \to \langle qcurve \rangle$. Fully measures the $\langle qcurve \rangle$ register, but keeps the type of the register quantum.
- Casting $\langle qcurve \rangle$ to $\langle curve \rangle$: The register is declared to be classical. If the value is not already classical, it is measured.
- Casting $\langle curve \rangle$ to $\langle qcurve \rangle$: The register is declared to be quantum, but is otherwise left unchanged.

[5] As argued by Zhandry [Zha25], an algebraic interpretation is problematic for some primitives, such as quantum money schemes, which require fully quantum games.

Algebraic games. A security game G can now have both classical oracles $\mathsf{O}_1, \ldots, \mathsf{O}_n$ and quantum oracles $\mathsf{QO}_1, \ldots, \mathsf{QO}_m$. We generally model quantum oracles as defined in Section 2.4, except that they can also do operations on quantum curves. We restrict to the following type of game:

1. The game never gives a quantum curve $\mathtt{Q}.\mathbf{E}$ as input to the adversary $\mathcal{A}$. However, $\mathcal{A}$ might output quantum curves.
2. Quantum oracles cannot output quantum curves. However, it may receive quantum curves as input from the adversary $\mathcal{A}$.
3. The game state is split into $S = (S_c, S_q)$. The classical oracles share the state S_c, while the quantum oracles have access to all of S.

In the spirit of the classical definition, we call G an *algebraic game* if it does not invoke decode. An example of an algebraic game with a (stateful) quantum oracle is presented in Fig. 8.

Algebraic algorithms. Say the quantum algorithm $\mathcal{A}$ gets the classical curves $\mathbf{E}_1, \ldots, \mathbf{E}_n$ as input. We say that $\mathcal{A}$ is an *algebraic* algorithm if:

1. When it outputs a classical curve $\mathbf{E}$, it also outputs a classical explanation $\varphi_{\mathsf{expl}} : \mathbf{E}_{\mathsf{end}} \to \mathbf{E}$ or $\varphi_{\mathsf{expl}} : \mathbf{E}_k \to \mathbf{E}$.
2. When it outputs a quantum curve $\mathtt{Q}.E$, it also outputs a superposition of valid explanations $\mathtt{Q}.X$ w.r.t. the inputs $\mathbf{E}_{\mathsf{end}}, \mathbf{E}_1, \ldots, \mathbf{E}_n$.

We call $\mathtt{Q}.X$ an explanation register. We write $[\mathtt{Q}.\mathbf{E}]$ for the joint output $(\mathtt{Q}.\mathbf{E}, \mathtt{Q}.X)$. When the algebraic algorithm outputs $[\mathtt{Q}.\mathbf{E}]$, the system's state is

$$\sum_{\mathbf{F}, \varphi_{\mathsf{expl}}} \alpha_{\mathbf{F}, \varphi_{\mathsf{expl}}} |\mathbf{F}\rangle_{\mathbf{E}} |\varphi_{\mathsf{expl}}\rangle \otimes |\ldots\rangle ,$$

where $|...\rangle$ denotes the state of other entangled registers, and each pair $(\mathbf{F}, \varphi_{\mathsf{expl}})$ is a classical curve and explanation.

As in the classical case, we want to emphasize that the algebraic game does not make use of and does not depend on the explanations. These are only used by the quantum algebraic reduction.

Remark 3.1. Similar to the quantum algebraic group action model [DHK+23], we restrict ourselves to well-behaved adversaries that always output correct explanations. Alternatively, we could have defined the games to compute whether the explanations are valid in superposition, and then measure the result. The adversary would then lose the game if the check failed.

Algebraic reductions. The definition of an algebraic reduction R is identical to the classical definition. If $\mathcal{A}$ is algebraic, then $\mathcal{B} := \mathsf{R}^{\mathcal{A}}$ must be algebraic.

The following lemma is useful for composing quantum algebraic algorithms.

Lemma 3.2. *Let $\mathbf{E}_1, \ldots, \mathbf{E}_m$ be curves and let $\mathcal{A}$ and $\mathcal{B}$ be quantum algebraic algorithms, where $\mathcal{B}$ gets as input $(\mathbf{E}_1, \ldots, \mathbf{E}_m)$ and runs adversary $\mathcal{A}$ as a sub-routine. Say $\mathcal{B}$ has explanations for the classical curves $\mathbf{F}_1, \ldots, \mathbf{F}_n$ and runs $\mathcal{A}$ on this input, to obtain*

$$[\mathtt{Q}.\mathbf{F}] \leftarrow \mathcal{A}(\mathbf{F}_1, \ldots, \mathbf{F}_n),$$

where $[\mathtt{Q}.\mathbf{F}]$ contains a valid quantum explanation with respect to $\mathcal{A}$'s inputs. Then $\mathcal{B}$ can efficiently compute a valid quantum explanation for $\mathtt{Q}.\mathbf{F}$ with respect to its own inputs $\mathbf{E}_1, \ldots, \mathbf{E}_m$ without performing any measurements.

Proof. The algorithm $\mathcal{B}$ has $[\mathbf{F}_i] = (\mathbf{F}_i, \varphi_i)$ with $\varphi_i : \mathbf{E}_j \to \mathbf{F}_i$ for some $j \in \{\mathsf{end}, 1, \ldots, m\}$. We denote the explanation register output by $\mathcal{A}$ as $\mathtt{Q}.X_{\mathcal{A}}$. We want to create an explanation register $\mathtt{Q}.X_{\mathcal{B}}$ that is valid with respect to $\mathcal{B}$'s input curves. Note that those curves, as well as those that $\mathcal{B}$ provides to $\mathcal{A}$, are classical.

When given $(\mathtt{Q}.\mathbf{F}, \mathtt{Q}.X_{\mathcal{A}})$, $\mathcal{B}$ initializes an empty output register. Let $\mathbf{F}_{\mathsf{end}} = \mathbf{E}_{\mathsf{end}}$. $\mathcal{B}$ applies a unitary that computes the following: For each $i \in \{\mathsf{end}, 1, \ldots, n\}$ it checks whether the domain of φ in $\mathtt{Q}.X_{\mathcal{A}}$ is $\mathbf{F}_i$ and adds $\varphi \circ \varphi_i$ to $\mathtt{Q}.X_{\mathcal{B}}$. Since $\mathtt{Q}.X_{\mathcal{A}}$ is a superposition over isogenies of the form $\varphi : \mathbf{F}_i \to \mathbf{F}$, the new register $\mathtt{Q}.X_{\mathcal{B}}$ is a superposition over isogenies $\varphi' : \mathbf{E}_j \to \mathbf{F}$. □

Remark 3.3 (Lifting reductions from the QROM *to the* AIM*).* If a reduction R in the typed ROM does not obliviously sample curves, it can by definition be lifted to a reduction in the AIM+ROM (the AIM with a random oracle).

For lifting reductions in the QROM, more care is needed. It could happen that the algorithm $\mathsf{R}^{\mathcal{A}}$ measures a quantum curve $\mathtt{Q}.E$ from the quantum queries to the random oracle, obtaining a classical curve $\mathbf{E}$. If $\mathsf{R}^{\mathcal{A}}$ outputs $\mathbf{E}$ as a classical curve, it needs to also output a classical explanation. This introduces an additional measurement, which was not handled in the analysis of the QROM reduction, and could irreversibly collapse the state of $\mathcal{A}$.

The trick to solve this is to measure $\mathtt{Q}.\mathbf{E}$, but keep its type as a quantum curve. That way, the associated explanation register does not have to be measured. We will see an example of this in the security proof of SQIsign in Section 5.

4 From the AGAM to the AIM

In this section, we show that any results in the Algebraic Group Action Model with Twists ($\mathsf{AGAM}^{\top}$) can be lifted automatically to the AIM. In other words, despite the limitations of the $\mathsf{AGAM}^{\top}$, results proved secure in $\mathsf{AGAM}^{\top}$ also hold in the more generic setting of the AIM.

To formalize this lifting, we first introduce an algebraic group action model that is explicitly isogeny-based: while the only instantiations of the group actions (with twists) come from oriented isogenies, the lifting would not work for potential instantiations of the $\mathsf{AGAM}^{\top}$ that are not from isogenies. The new model, called O-AGAM, also generalizes the classic $\mathsf{AGAM}^{\top}$: while the $\mathsf{AGAM}^{\top}$ only worked with origin objects that are their own twist, we lift this restriction.

Hence, any result for isogeny-based constructions in the $\mathsf{AGAM}^{\top}$ automatically holds in the O-AGAM, and by the result we show in Section 4.2, also in the AIM.

4.1 The O-AGAM

We consider an effective group action (EGA) instantiated in the orientation framework. Let $\mathfrak{O}$ be an order in a quadratic imaginary field K with conductor f. We consider the action of $G = \mathrm{Cl}(\mathfrak{O})$ on the set $\mathcal{X} = \mathsf{SS}_{\mathfrak{O}}(p)$. The origin $x_0 = (\mathbf{E}_0, \iota_0) \in \mathcal{X}$ is some starting curve of known endomorphism ring. We enforce two restrictions on the parameters:

1. The group action has one orbit and it is non-empty. By Theorem 2.7, this is equivalent to p being ramified in K and p not dividing the conductor f. We make this restriction to ensure that we can efficiently test for membership in $\mathcal{X}$. If the action has two orbits, there is to best of our knowledge no efficient algorithm for checking which orbit an oriented curve belongs to.
2. The conductor f is polylog(p)-smooth. This is a technical requirement for the lifting result. In practice this is not an issue. The $\mathsf{AGAM}^{\top}$ is based on CSIDH [CLM+18] and CSURF [CD20], where the conductor is either 1 or 2.

We consider the following algebraic group action model for $(G, \mathcal{X}, x_0, \star)$, which we call the O-AGAM:

– An isomorphism class $(\mathbf{E}, \iota) \in \mathcal{X}$ is represented by its unique representative, as a $(\langle curve \rangle, \langle bits \rangle)$ pair.

– The games in this model only work with oriented curves $(\mathbf{E}, \iota) \in \mathcal{X}$. Elliptic curves do not appear without an orientation.

– To compute $(\mathbf{F}, \iota_\mathbf{F}) = \mathfrak{a} \star (\mathbf{E}, \iota_\mathbf{E})$ in typed pseudocode, we define

$$\mathsf{isogeny}_\mathfrak{O}\ (\langle curve \rangle\ \mathbf{E}, \langle bits \rangle\ (\iota_\mathbf{E}, \mathfrak{a})) \to (\langle curve \rangle\ \mathbf{F}, \langle bits \rangle\ \iota_\mathbf{F}).$$

It checks that ι is a primitive $\mathfrak{O}$-orientation of $\mathbf{E}$, and if so, computes the action of the $\mathfrak{O}$-ideal $\mathfrak{a}$. If the input is not valid, it outputs $\perp$.

– The twist $\tau(\mathbf{E}, \iota) = (\mathbf{E}, \bar{\iota})$ can be computed directly on the bit-representation.

– Algebraic games are defined as for the AIM, except that the only isogeny operation allowed is $\mathsf{isogeny}_\mathfrak{O}$. An algebraic game in the O-AGAM is still an algebraic game in the AIM. We emphasize that the adversary can only receive classical oriented curves as input from the game.

– Algebraic algorithms are defined as follows: Assume an algorithm $\mathcal{A}$ has received as input the oriented curves $(\mathbf{E}_1, \iota_1), \ldots, (\mathbf{E}_n, \iota_n) \in \mathcal{X}$. We say that $\mathcal{A}$ is $\mathfrak{O}$-algebraic, if whenever it outputs some $(\mathbf{F}, \iota) \in \mathcal{X}$, it also outputs an explanation $(\mathfrak{a}, b) \in G \times \{0, 1\}$ such that for some $j \in \{0, \ldots, n\}$,

$$\mathsf{isogeny}_\mathfrak{O}(\mathbf{E}_j, (\iota_j, \mathfrak{a})) = \begin{cases} (\mathbf{F}, \iota) & \text{if } b = 0, \\ (\mathbf{F}, \bar{\iota}) & \text{if } b = 1. \end{cases}$$

4.2 Lifting result

In this section, we prove that a reduction in the O-AGAM can be lifted to the AIM. To do this, we need to be able to convert between explanations in the two models. Converting from an explanation in the O-AGAM (i.e., an ideal) to an explanation in the AIM (i.e., an isogeny) is done using the Clapoti algorithm [PR23].

Lemma 4.1 ([PR23, Theorem 2.9]). *Let $(\mathbf{E}, \iota_\mathbf{E}), (\mathbf{F}, \iota_\mathbf{F}) \in \mathsf{SS}_\mathfrak{O}(p)$ be oriented curves and let $\mathfrak{a}$ be an invertible $\mathfrak{O}$-ideal such that $[\mathfrak{a}] \star (\mathbf{E}, \iota_\mathbf{E}) = (\mathbf{F}, \iota_\mathbf{F})$. There exists an algorithm that computes*

$$\varphi_\mathfrak{a} : (\mathbf{E}, \iota_\mathbf{E}) \to (\mathbf{F}, \iota_\mathbf{F})$$

in (classical) expected polynomial time in $\log p$, $\log \mathrm{N}(\mathfrak{a})$ and $\log|\mathrm{disc}(\mathfrak{O})|$.

Given an O-AGAM-explanation $(\mathfrak{a}, b)$, we simply ignore b and compute $\varphi_\mathfrak{a} : \mathbf{E} \to \mathbf{F}$ as the AIM-explanation.

Next, we consider the problem of converting an explanation in the AIM to an explanation in the O-AGAM. Given $(\mathbf{E}, \iota_\mathbf{E}), (\mathbf{F}, \iota_\mathbf{F}) \in \mathsf{SS}_\mathfrak{O}(p)$ and an isogeny $\varphi : \mathbf{E} \to \mathbf{F}$, we need to compute an $\mathfrak{O}$-ideal $\mathfrak{a}$ such that $\mathfrak{a} \star (\mathbf{E}, \iota_\mathbf{E}) = (\mathbf{F}, \iota_\mathbf{F})$.

Lemma 4.2. *Let $(\mathbf{E}, \iota_\mathbf{E}), (\mathbf{F}, \iota_\mathbf{F}) \in \mathsf{SS}_\mathfrak{O}(p)$ be oriented curves and let $\varphi : \mathbf{E} \to \mathbf{F}$ be an isogeny. Given $\mathbf{E}, \mathbf{F}$ and an efficient representation of $\iota_\mathbf{E}, \iota_\mathbf{F}$ and φ, we can compute an efficient representation of a K-oriented isogeny*

$$\psi : (\mathbf{E}, \iota_\mathbf{E}) \to (\mathbf{F}, \iota_\mathbf{F}) \quad \text{or} \quad \psi : (\mathbf{E}, \iota_\mathbf{E}) \to (\mathbf{F}, \overline{\iota_\mathbf{F}})$$

in (classical) polynomial time in $\log p$, $\log \deg(\varphi)$ and $\log|\mathrm{disc}(\mathfrak{O})|$.

Proof. The representation of $\iota_\mathbf{E}$ includes the encoding of a generator ω of $\mathfrak{O}$, such that $\mathfrak{O} = \mathbb{Z}[\omega]$. Let $\alpha = 2\omega - \mathrm{Tr}(\omega)$. We have $\alpha \neq 0$, $\mathrm{Tr}(\alpha) = 0$ and $\alpha^2 = -\mathrm{N}(\alpha) \in \mathbb{Z}$. Define $\sigma = \iota_\mathbf{F}(\alpha) \circ \varphi + \varphi \circ \iota_\mathbf{E}(\alpha) : \mathbf{E} \to \mathbf{F}$.

Case 1: $\sigma \neq 0$. We claim that $\sigma : (\mathbf{E}, \iota_\mathbf{E}) \to (\mathbf{F}, \iota_\mathbf{F})$ is K-oriented isogeny.

$$\iota_\mathbf{F}(\alpha) \circ \sigma = [\alpha^2] \circ \varphi + \iota_\mathbf{F}(\alpha) \circ \varphi \circ \iota_\mathbf{E}(\alpha) = \sigma \circ \iota_\mathbf{E}(\alpha).$$

By substitution, we get $\iota_{\mathbf{F}}(\omega) \circ ([2] \circ \sigma) = ([2] \circ \sigma) \circ \iota_{\mathbf{E}}(\omega)$. This implies that

$$\iota_{\mathbf{F}}(\omega) = \left(([2] \circ \sigma) \circ \iota_{\mathbf{E}}(\omega) \circ (\widehat{[2] \circ \sigma})\right) \otimes \frac{1}{\deg([2] \circ \sigma)} = (\sigma \circ \iota_{\mathbf{E}}(\omega) \circ \widehat{\sigma}) \otimes \frac{1}{\deg(\sigma)},$$

showing that $[2] \circ \sigma$, and thereby σ, is a K-oriented isogeny $(\mathbf{E}, \iota_{\mathbf{E}}) \to (\mathbf{F}, \iota_{\mathbf{F}})$.

Case 2: $\sigma = 0$. We claim that we can output φ.

$$\varphi \circ \iota_{\mathbf{E}}(\alpha) = -\iota_{\mathbf{F}}(\alpha) \circ \varphi = \iota_{\mathbf{F}}(\overline{\alpha}) \circ \varphi$$

Note that $\overline{\alpha} = 2\overline{\omega} - \mathrm{Tr}(\omega)$. By substitution, we obtain that $[2] \circ \varphi$, and thereby φ, is a K-oriented isogeny $(\mathbf{E}, \iota_{\mathbf{E}}) \to (\mathbf{F}, \overline{\iota_{\mathbf{F}}})$. □

Lemma 4.3. *Let $(\mathbf{E}, \iota_{\mathbf{E}}), (\mathbf{F}, \iota_{\mathbf{F}}) \in \mathsf{SS}_{\mathfrak{O}}(p)$ be oriented curves and let ℓ be a prime that does not divide the conductor of $\mathfrak{O}$. Let $\varphi : (\mathbf{E}, \iota_{\mathbf{E}}) \to (\mathbf{F}, \iota_{\mathbf{F}})$ be a cyclic K-oriented isogeny of degree ℓ^t for some $t \in \mathbb{Z}_{>0}$. Given ℓ, the curves $\mathbf{E}$ and $\mathbf{F}$, and an efficient representation of $\iota_{\mathbf{E}}, \iota_{\mathbf{F}}$ and φ, we can compute an invertible $\mathfrak{O}$-ideal $\mathfrak{a}$ such that*

$$(\mathbf{F}, \iota_{\mathbf{F}}) = [\mathfrak{a}] \star (\mathbf{E}, \iota_{\mathbf{E}}) \text{ and } \mathrm{N}(\mathfrak{a}) = \ell^t$$

in (classical) polynomial time in $\log p$, $\log \ell$, t and $\log|\mathrm{disc}\, \mathfrak{O}|$.

Proof. Consider any factorization of φ into degree ℓ isogenies

$$\varphi : (\mathbf{F}_0, \iota_0) \xrightarrow{\varphi_1} (\mathbf{F}_1, \iota_1) \xrightarrow{\varphi_2} \dots \xrightarrow{\varphi_t} (\mathbf{F}_t, \iota_t),$$

with $(\mathbf{F}_0, \iota_0) = (\mathbf{E}, \iota_{\mathbf{E}})$ and $(\mathbf{F}_t, \iota_t) = (\mathbf{F}, \iota_{\mathbf{F}})$. We will prove that φ is horizontal, i.e. that every φ_i is horizontal.

We first handle the case when $\ell \neq p$. We claim that if some φ_i is descending, then because φ is cyclic, it follows that every subsequent step is also descending. If $i < t$, φ_{i+1} could not be horizontal, as there are no horizontal isogenies once we descend. Additionally, φ_{i+1} could not be ascending, as φ is cyclic, and the only ascending isogeny backtracks φ_i. Hence, φ_{i+1} would have to be descending. By induction, we see that from φ_i onward, every step must be descending. When the domain and codomain of φ are oriented by the same order $\mathfrak{O}$, we conclude that every φ_i must be horizontal. Furthermore, ℓ cannot be inert in K; otherwise, there would be no horizontal ℓ-isogenies.

When $\ell = p$, the Frobenius isogeny π is the only isogeny of degree p (up to post-composition with an isomorphism). π is horizontal and satisfies that π^2 is divisible by p. Because φ is cyclic, we must have $t = 1$ and $\varphi = \alpha \circ \pi$, where α is a K-oriented isomorphism.

Let us now discuss how to compute an invertible $\mathfrak{O}$-ideal $\mathfrak{a}$ corresponding to φ. Initially, we set $\mathfrak{a} = (1)$. There are one or two prime $\mathfrak{O}$-ideals $\mathfrak{p}$ of norm ℓ, corresponding to the possible horizontal ℓ-isogenies. These ideals are invertible, as their norm is coprime to the conductor. We use Lemma 4.1 to convert $\mathfrak{p}$ to an isogeny ψ in efficient representation, then attempt to divide φ by ψ. If it is divisible, i.e. $\varphi = \varphi' \circ \psi$, we set $\mathfrak{a} \coloneqq \mathfrak{a} \cdot \mathfrak{p}$. We then continue the trial division with φ' until $\deg \varphi' = 1$. In the end, we recover an invertible ideal $\mathfrak{a}$ such that $[\mathfrak{a}] \star (\mathbf{E}, \iota_{\mathbf{E}}) = (\mathbf{F}, \iota_{\mathbf{F}})$ and $\mathrm{N}(\mathfrak{a}) = \ell^t$. □

Lemma 4.4. *Let $(\mathbf{E}, \iota_{\mathbf{E}}), (\mathbf{F}, \iota_{\mathbf{F}}) \in \mathsf{SS}_{\mathfrak{O}}(p)$ be oriented curves and let $\ell \neq p$ be a prime that divides the conductor f of $\mathfrak{O}$. Let $\varphi : (\mathbf{E}, \iota_{\mathbf{E}}) \to (\mathbf{F}, \iota_{\mathbf{F}})$ be a cyclic K-oriented isogeny of degree ℓ^t for some $t \in \mathbb{N}$. Given ℓ, the curves $\mathbf{E}$ and $\mathbf{F}$, an efficient representation of $\iota_{\mathbf{E}}, \iota_{\mathbf{F}}$ and φ, and the structure of the class group $\mathrm{Cl}(\mathfrak{O})$, we can compute an invertible $\mathfrak{O}$-ideal $\mathfrak{a}$ such that*

$$(\mathbf{F}, \iota_{\mathbf{F}}) = [\mathfrak{a}] \star (\mathbf{E}, \iota_{\mathbf{E}})$$

in (classical) polynomial time in $\log p$, ℓ, t and $\log|\mathrm{disc}\, \mathfrak{O}|$.

Proof. Let r be the largest integer such that ℓ^r divides the conductor of $\mathfrak{O}$ and let $s = t - 2r$. We claim that φ factors as $\varphi = \varphi_{\mathsf{d}} \circ \varphi_{\mathsf{h}} \circ \varphi_{\mathsf{a}}$, where φ_{a} is the composition of r ascending ℓ-isogenies, φ_{h} is the composition of s horizontal ℓ-isogenies, and φ_{d} is the composition of r descending ℓ-isogenies. As argued in the previous proof, once a cyclic K-oriented ℓ^t-isogeny has a descending step, all the subsequent steps must be descending. Hence, the first r steps of φ must be ascending. If $s = t - 2r > 0$, the next s steps must be horizontal. Finally, the last r steps must be descending, to arrive back at a curve primitively oriented by $\mathfrak{O}$.

We will now describe how we compute $\mathfrak{a}$. We begin by splitting φ into ℓ-isogenies, so that we recover $\varphi_{\mathsf{a}}, \varphi_{\mathsf{h}}$ and φ_{d}. This can be accomplished by trial division, where we enumerate through the possible ℓ-isogenies for each step. This takes time polynomial in ℓ and t.

Write $\varphi_{\mathsf{h}} : (\mathbf{E}', \iota'_{\mathbf{E}}) \to (\mathbf{F}', \iota'_{\mathbf{F}})$ and let $\mathfrak{O}'$ be the order that primitively orients these curves. Using Lemma 4.3, we obtain an invertible $\mathfrak{O}'$-ideal $\mathfrak{b}$ corresponding to φ_{h} such that $(\mathbf{F}', \iota'_{\mathbf{F}}) = [\mathfrak{b}] \star (\mathbf{E}', \iota'_{\mathbf{E}})$ and $\mathrm{N}(\mathfrak{b}) = \deg(\varphi_{\mathsf{h}}) = \ell^s$. We then compute an ideal $\mathfrak{c}$ equivalent to $\mathfrak{b}$ of norm coprime to f. These steps take polynomial time in $\log p$, $t \log \ell$, and $\log |\operatorname{disc} \mathfrak{O}|$.

Finally, given an invertible ideal $\mathfrak{c}$ of norm coprime to f such that $(\mathbf{F}', \iota'_{\mathbf{F}}) = [\mathfrak{c}] \star (\mathbf{E}', \iota'_{\mathbf{E}})$, there is a recursive algorithm described in [FHL+25] to compute an invertible $\mathfrak{O}$-ideal $\mathfrak{a}$ such that $(\mathbf{F}, \iota_{\mathbf{F}}) = [\mathfrak{a}] \star (\mathbf{E}, \iota_{\mathbf{E}})$. This algorithm requires the structure of $\mathrm{Cl}(\mathfrak{O})$. It runs in classical polynomial time in ℓ, r, $\log \mathrm{N}(\mathfrak{c})$ and $\log |\operatorname{disc} \mathfrak{O}|$. □

Combining the last three lemmas, we obtain an algorithm to convert an AIM explanation to an O-AGAM explanation when the conductor of $\mathfrak{O}$ is smooth.

Lemma 4.5. *Let $(\mathbf{E}, \iota_{\mathbf{E}}), (\mathbf{F}, \iota_{\mathbf{F}}) \in \mathsf{SS}_{\mathfrak{O}}(p)$ be oriented curves and let $\varphi : \mathbf{E} \to \mathbf{F}$ be an isogeny. Given $\mathbf{E}$, $\mathbf{F}$ and an efficient representation of $\iota_{\mathbf{E}}, \iota_{\mathbf{F}}$ and φ, we can compute an invertible $\mathfrak{O}$-ideal $\mathfrak{a}$ and a bit $b \in \{0, 1\}$ such that*

$$(\mathbf{F}, \iota_{\mathbf{F}}) = \begin{cases} [\mathfrak{a}] \star (\mathbf{E}, \iota_{\mathbf{E}}) & \text{if } b = 0 \\ \tau([\mathfrak{a}] \star (\mathbf{E}, \iota_{\mathbf{E}})) & \text{if } b = 1, \end{cases}$$

in expected quantum polynomial time in $\log p$, $\log d$ and $\log|\operatorname{disc} \mathfrak{O}|$.

Proof. We use Lemma 4.2 to obtain a K-oriented isogeny ψ such that $\psi : (\mathbf{E}, \iota_{\mathbf{E}}) \to (\mathbf{F}, \iota_{\mathbf{F}})$ or $\psi : (\mathbf{E}, \iota_{\mathbf{E}}) \to (\mathbf{F}, \iota_{\mathbf{F}})$. In the former case we set $b = 0$, and in the latter case $b = 1$. Next, we use a quantum algorithm to factor $\deg \psi = \prod_{i=1}^{k} \ell_i$. By repeatedly dividing ψ with each $[\ell_i]$, we obtain its cyclic component ψ'. We then decompose ψ' as $\psi' = \psi'_k \circ \cdots \circ \psi'_1$, where $\psi'_i : (\mathbf{F}_{i-1}, \iota_{i-1}) \to (\mathbf{F}_i, \iota_i)$ has degree $\ell_i^{t_i}$ for some $t_i \in \mathbb{Z}_{\geq 0}$. For each i with $t_i > 0$, we compute an invertible ideal $\mathfrak{a}_i$ such that $(\mathbf{F}_i, \iota_i) = [\mathfrak{a}_i] \star (\mathbf{F}_{i-1}, \iota_{i-1})$. If $\ell_i \nmid f$, we do this using the algorithm in Lemma 4.3. Otherwise, if $\ell_i \mid f$, we use the algorithm in Lemma 4.4. The runtime of this algorithm depends polynomially on ℓ_i, which is why we need the assumption that f is polylog(p) smooth. The algorithm also needs as input the structure of the class group $\mathrm{Cl}(\mathfrak{O})$, which can be computed in quantum polynomial time in $\log |\operatorname{disc} \mathfrak{O}|$ [Hal05]. In the end, we obtain $\mathfrak{a} = \mathfrak{a}_k \cdot \cdots \cdot \mathfrak{a}_1$. □

Combining these results, we obtain the lifting result. In Appendix D, we prove an analogous result for quantum games. The proof is the essentially the same, except that the conversions are performed in superposition.

Theorem 4.6. *Let G be a classical algebraic game in the O-AGAM. Let $\mathcal{A}$ be an algebraic algorithm for G in the AIM (resp. O-AGAM). Let n be an upper bound on the number of curves output by $\mathcal{A}$, and d an upper bound on the degree (resp. norm) of their explanations. Finally, let $t \in \mathbb{N}$. Then there is an algebraic algorithm $\mathcal{B}$ for G in the O-AGAM (resp. AIM), with*

$$\begin{aligned} \mathsf{Adv}^{\mathsf{G}}(\mathcal{B}) &\geq \mathsf{Adv}^{\mathsf{G}}(\mathcal{A}) - n \cdot 2^{-t}, \\ \mathsf{Time}^{\mathsf{G}}(\mathcal{B}) &\leq \mathsf{Time}^{\mathsf{G}}(\mathcal{A}) + nt \cdot \mathsf{poly}(\log p, \log d, \log|\operatorname{disc} \mathfrak{O}|). \end{aligned}$$

If $\mathcal{A}$ is AIM-algebraic, then $\mathcal{B}$ is a quantum algorithm.

Proof. The idea is to convert the explanations $\mathcal{A}$ outputs using the previous algorithms. If $\mathcal{A}$ is O-AGAM-algebraic, we use Lemma 4.1, and if $\mathcal{A}$ is AIM-algebraic, we use Lemma 4.5.

In both cases, we have a conversion algorithm C, running in expected time $\mathbb{E}[T] = \mathsf{poly}(\log p, \log d, \log|\operatorname{disc}(\mathfrak{O})|)$. We convert C to a PPT algorithm D. By Markov's inequality, $\Pr\left[T > 2\cdot\mathbb{E}[T]\right] \leq 1/2$. Let C' be the algorithm C modified to abort after time $2\cdot\mathbb{E}[T]$. Let D be the algorithm that runs C' at most t times, each time with new randomness. Then D aborts with probability $\leq 2^{-t}$.

We define $\mathcal{B}$ to internally run $\mathcal{A}$, and whenever $\mathcal{A}$ outputs a curve and an explanation, $\mathcal{B}$ runs D to compute the new explanation. If D aborts, then so does $\mathcal{B}$. Let abort_i be the event that D aborts when converting the i-th explanation.

$$\Pr\left[\mathsf{G}(\mathcal{B})=1\right] = \Pr\left[\mathsf{G}(\mathcal{A})=1 \wedge \bigwedge_{i=1}^{n} \neg\mathsf{abort}_i\right] \geq \Pr\left[\mathsf{G}(\mathcal{A})=1\right] - \sum_{i=1}^{n}\Pr\left[\mathsf{abort}_i\right]$$

□

These results allow us to lift any algebraic reduction in the O-AGAM to an algebraic reduction in the AIM. Given an AIM-algebraic algorithm $\mathcal{A}$ for G, we convert it to an O-AGAM-algebraic algorithm $\mathcal{B}$, then convert $R^{\mathcal{B}}$ to an AIM-algebraic algorithm.

5 Quantum security of SQIsign

In this section, we analyze the quantum EUF-CMA-security of SQIsign in the AIM + QROM. First, in Section 5.1, we recall the Fiat–Shamir with hints framework from [ABD+25] that was introduced to analyze the classical security of Fiat–Shamir signatures like SQIsign. Then, in Section 5.2, we introduce a new security game for analyzing quantum security in the framework. Finally, in Section 5.3, we show that for SQIsign, hint-EndRing can be reduced to this game in the AIM.

5.1 Background on the Fiat–Shamir with hints framework and SQIsign

The Fiat–Shamir framework was introduced in [ABD+25] to analyze the (classical) security of SQIsign and related Fiat–Shamir signatures. The framework captures the scenario where some auxiliary input, *a hint*, is needed for simulation. This hint is sampled from a hint distribution.

Definition 5.1 (Hint distribution). *Let $R \subseteq \mathcal{X}\times\mathcal{W}$ be a relation. A hint distribution $\mathcal{H}$ for R is a collection of distributions $\mathcal{H} = \{\mathcal{H}_x\}_{x\in\mathcal{X}}$, where $\mathcal{H}_x : \mathsf{HintSet}_x \to [0,1]$ and the elements (i.e. the hints) of $\mathsf{HintSet}_x$ are efficiently representable in $|x|$. The distribution $\mathcal{H}_x$ need not be efficiently sampleable.*

Definition 5.2 (Hint-assisted wHVZK). *Let Σ be a sigma protocol for a relation R and let $\mathcal{H}$ be a hint distribution for R. Assume that Σ comes with a PPT algorithm, called the simulator. We define hint-assisted weak-honest-verifier zero-knowledge by the family of games q-hint-wHVZK in Fig. 4. The advantage of an adversary $\mathcal{A}$ playing the game q-hint-wHVZK is*

$$\mathsf{Adv}^{q\text{-hint-wHVZK}}_{\Sigma,\mathcal{H}}(\mathcal{A}) := \left|\Pr\left[q\text{-hint-wHVZK}_{\Sigma,\mathcal{H},0}(\mathcal{A})\to 1\right] - \Pr\left[q\text{-hint-wHVZK}_{\Sigma,\mathcal{H},1}(\mathcal{A})\to 1\right]\right|.$$

To achieve security with hint-wHVZK, we need that the relation is hard even if the adversary is given a polynomial number of hints.

Definition 5.3 (Hard relation with hints). *Let R be a relation and let $\mathcal{H}$ be a hint distribution for R. We call the pair $(R,\mathcal{H})$ a relation with hints. For every $q\in\mathbb{N}$, the relation with hints has an associated game, where the adversary $\mathcal{A}$ is given a statement and q hints and has to find an associated witness. The advantage of $\mathcal{A}$ in this game is*

$$\mathsf{Adv}^{q\text{-hint-rel}}_{(R,\mathcal{H})}(\mathcal{A}) := \Pr\left[(x,w^*)\in R \;\middle|\; \begin{array}{r}(x,w)\leftarrow \mathsf{Gen}_R(1^\lambda),\\ h_1,\ldots,h_q\leftarrow\mathcal{H}_x,\\ w^*\leftarrow\mathcal{A}(x,h_1,\ldots,h_q)\end{array}\right].$$

Game q-hint-wHVZK$_{\Sigma,\mathcal{H},b}(\mathcal{A})$

00 $(x, w) \leftarrow \mathsf{Gen}_R(1^\lambda)$
01 $(\pi_1, \ldots, \pi_q) \leftarrow \mathsf{O}_b$
02 $b' \leftarrow \mathcal{A}(x, \pi_1, \ldots, \pi_q)$
03 **return** b'

Procedure O_0

04 **for** $i = 1, \ldots, q$ **do**
05 $(\mathsf{com}_i, \mathsf{state}_i) \leftarrow \mathcal{P}_1(x, w)$
06 $\mathsf{chl}_i \xleftarrow{\$} \mathcal{C}$
07 $\mathsf{rsp}_i \leftarrow \mathcal{P}_2(\mathsf{state}_i, \mathsf{chl}_i)$
08 $\pi_i := (\mathsf{com}_i, \mathsf{chl}_i, \mathsf{rsp}_i)$
09 **return** $(\pi_1, \ldots, \pi_q)$

Procedure O_1

10 **for** $i = 1, \ldots, q$ **do**
11 $h_i \leftarrow \mathcal{H}_x$
12 $\pi_i \leftarrow \mathcal{S}(x, h_i)$
13 **return** $(\pi_1, \ldots, \pi_q)$

Fig. 4: hint-wHVZK game for a Σ-protocol Σ with simulator $\mathcal{S}$. The game is defined relative to a bit $b \in \{0, 1\}$ and the number of transcripts q.

Algorithm $\mathcal{H}^{\mathsf{SQI}}_{\mathbf{E}}$

00 Sample an integer d from a weighted distribution on the interval $[1, 2^{e_{\mathsf{rsp}}}]$ where each integer n, with prime factorization $n = \prod_{i=1}^{t} p_i^{e_i}$, has weight $\prod_{i=1}^{t}(p_i + 1)^{e_i}$
01 Sample an isogeny $\psi_1' : \mathbf{E} \to \mathbf{E_1'}$ uniformly among the (possibly non-cyclic) isogenies from $\mathbf{E}$ of degree d
02 Write $\psi_1 : \mathbf{E} \to \mathbf{E_1}$ for the cyclic component of ψ_1'.
03 Write $\deg \psi_1 = 2^n d'$ with d' odd
04 Sample an isogeny $\psi_2 : \mathbf{E}_1 \to \mathbf{E}_2$ uniformly among the cyclic isogenies from $\mathbf{E}_1$ of degree $2^{e_{\mathsf{rsp}}-n} - d'$
05 **return** $h = (\psi_1, \psi_2)$

Fig. 5: The (pushable) hint distribution for SQIsign from [ABD+25]. All the isogenies are given in the unique 8D-representation.

For the purposes of our analysis, we recall some key aspects of SQIsign. The Fiat-Shamir signature is obtained from a Σ-protocol Σ_{SQI} for the OneEnd relation. The prime p is of the form $p = c \cdot 2^e - 1$, where c is a small odd cofactor. We decompose the exponent as $e = e_{\mathsf{chl}} + e_{\mathsf{rsp}}$, where $2^{e_{\mathsf{chl}}}$ is the size of the challenge space and $2^{e_{\mathsf{rsp}}}$ is an upper bound on the degree of the response. For a full description of SQIsign, we refer the reader to [BDD+24, AAA+25].

To analyze SQIsign in the Fiat–Shamir with hints framework, [ABD+25] introduced the hint distribution $\mathcal{H}^{\mathsf{SQI}}$ in Fig. 5. We say that hints from this hint distribution are *pushable*. Given a hint h for $\mathbf{E}$ and the efficient representation of a cyclic 2^k-isogeny $\sigma : \mathbf{E} \to \mathbf{E}'$, there is a polynomial time algorithm PushHint that outputs a hint h' for $\mathbf{E}'$. If $h \leftarrow \mathcal{H}^{\mathsf{SQI}}_{\mathbf{E}}$, then h' is distributed as a hint from $\mathcal{H}^{\mathsf{SQI}}_{\mathbf{E}'}$.

Game q-hint-dist$_b(\mathcal{A})$

00 $(\mathbf{E}, I) \leftarrow \mathsf{Gen}_{R_{\mathsf{SQI}}}$
01 $s_1, \ldots, s_q \leftarrow \mathsf{O}_b$
02 $b' \leftarrow \mathcal{A}(\mathbf{E}, s_1, \ldots, s_q)$
03 **return** b'

Procedure O_0

04 $\mathsf{chl} \xleftarrow{\$} \{0, \ldots, 2^{e_{\mathsf{chl}}} - 1\}$
05 Let $\varphi_{\mathsf{chl}} : \mathbf{E} \to \mathbf{E}_1$ with kernel $\langle P + [\mathsf{chl}]Q \rangle$, where P, Q is a deterministic basis for $\mathbf{E}[2^e]$
06 $h \leftarrow \mathcal{H}^{\mathsf{SQI}}_{\mathbf{E}}$
07 $h' \leftarrow \mathsf{PushHint}(\mathbf{E}, h, \varphi_{\mathsf{chl}})$
08 **return** (chl, h')

Procedure O_1

09 $\mathsf{chl} \xleftarrow{\$} \{0, \ldots, 2^{e_{\mathsf{chl}}} - 1\}$
10 Let $\varphi_{\mathsf{chl}} : \mathbf{E} \to \mathbf{E}_1$ with kernel $\langle P + [\mathsf{chl}]Q \rangle$, where P, Q is a deterministic basis for $\mathbf{E}[2^e]$
11 Sample an isogeny $\varphi_1 : \mathbf{E}_1 \to \mathbf{E}_2$ s.t.:
12 1. $\mathbf{E_2}$ is sampled from the stationary distr. on $\mathsf{SS}(p)$
13 2. The conditional distribution of φ_1 given $\mathbf{E}_2$ is uniform among isogenies $\mathbf{E}_1 \to \mathbf{E}_2$ of degree $< 2^{e_{\mathsf{rsp}}}$
14 Decompose $\varphi_1 = [m] \circ \varphi_1'$ with φ_1' cyclic
15 Write $\deg(\varphi_1) = 2^n d'$ with d' odd
16 Sample an isogeny $\varphi_2 : \mathbf{E}_2 \to \mathbf{E}_3$ uniformly among the cyclic isogenies from $\mathbf{E}_2$ of degree $2^{e_{\mathsf{rsp}}-n} - d'$
17 **return** $(\mathsf{chl}, (\varphi_1', \varphi_2))$

Fig. 6: The hint-dist game for SQIsign, defined relative to a bit $b \in \{0, 1\}$ and to the number of oracle queries q. All the isogenies are given in the unique 8D-representation. The hint-wHVZK of SQIsign relies on the output of the procedures being computationally indistinguishable.

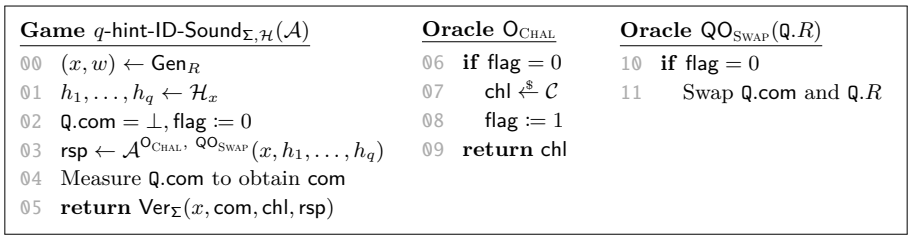

Fig. 7: hint-ID-Sound game for a Σ-protocol Σ and a hint distribution $\mathcal{H}$, defined relative to the number of hints q.

Classically, [ABD+25] shows that SQIsign is EUF-CMA-secure assuming the hardness of two new problems, hint-EndRing and hint-dist.

Definition 5.4 (q-hint-EndRing)**.** *Given a curve* $\mathbf{E}$ *sampled from the stationary distribution on* $\mathsf{SS}(p)$ *and* q *hints* $h_1, \ldots, h_q \leftarrow \mathcal{H}_{\mathbf{E}}^{\mathsf{SQI}}$*, find four endomorphisms in efficient representation that form a basis of* $\mathrm{End}(\mathbf{E})$ *as a lattice.*

The q-hint-OneEnd game is defined in the same way, except that the adversary only has to output a single non-scalar endomorphism. The second problem is hint-dist, which is needed for the hint-wHVZK of SQIsign.

Definition 5.5 (hint-dist)**.** *We define the game* q-hint-dist *using the oracles* O_0 *and* O_1 *defined in Fig. 6. The adversary* $\mathcal{A}$ *is given* $\mathbf{E}$ *and* q *from* O_b*, and wins if it outputs* b*. The advantage of* $\mathcal{A}$ *is*

$$\mathsf{Adv}^{q\text{-hint-dist}}(\mathcal{A}) := \left|\Pr\left[q\text{-hint-dist}_0(\mathcal{A}) \to 1\right] - \Pr\left[q\text{-hint-dist}_1(\mathcal{A}) \to 1\right]\right|.$$

Finally, we recall some useful results. They were originally stated only for classical adversaries, but the same proof holds for quantum adversaries as well.

Lemma 5.6 ([ABD+25])**.** *For any (quantum)* q-hint-wHVZK *adversary* $\mathcal{A}$ *for* Σ_{SQI}*, there is a classical algorithm* $\mathcal{B}$ *and a (quantum) algorithm* $\mathcal{D}$ *such that*

$$\mathsf{Adv}^{q\text{-hint-wHVZK}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathsf{rel}}_{R_{\mathsf{SQI}}}(\mathcal{B}) + \mathsf{Adv}^{q\text{-hint-dist}}(\mathcal{D}) + \frac{q}{\sqrt{p}}.$$

The running time of $\mathcal{D}$ *is about that of* $\mathcal{A}$*, while for* $\mathcal{B}$ *it is polynomial in* $\log p$*.*

Lemma 5.7 ([ABD+25])**.** *In* Σ_{SQI}*, the public key curve and the commitment curve are sampled from a distribution with statistical distance at most* $\frac{1}{2}p^{-1/2}$ *from the stationary distribution on* $\mathsf{SS}(p)$*. Additionally, we have* $\mathsf{MinEnt}(\Sigma_{\mathsf{SQI}}) \leq p^{-1/2}$*.*

5.2 A new security game for the Fiat–Shamir with hints framework

Toward the goal of analyzing the quantum security of signatures in the Fiat–Shamir with hints framework, we introduce a new security game.

Definition 5.8 (hint-ID-Sound)**.** *Let* Σ *be a* Σ*-protocol for a relation* R *and let* $\mathcal{H}$ *be a hint distribution. We define* hint-ID-Sound *by the family of games* q-hint-ID-Sound *in Fig. 7.*

The game q-hint-ID-Sound is very similar to the standard soundness game for Σ-protocols. The adversary sends a commitment com, receives a challenge chl, then has to produce a response rsp. There are two differences: 1. the adversary gets q hints as auxiliary input, and 2. com can be a quantum state. To obtain a single-stage game,[6] we model this using the quantum oracle

[6] The example from Fig. 3 is actually very similar to this game, but there the adversary is two-stage, i.e., it is split into $\mathcal{A}_1$ and $\mathcal{A}_2$. Here, we re-write the game using only oracles. This way we do not have to make the adversary's quantum state explicit.

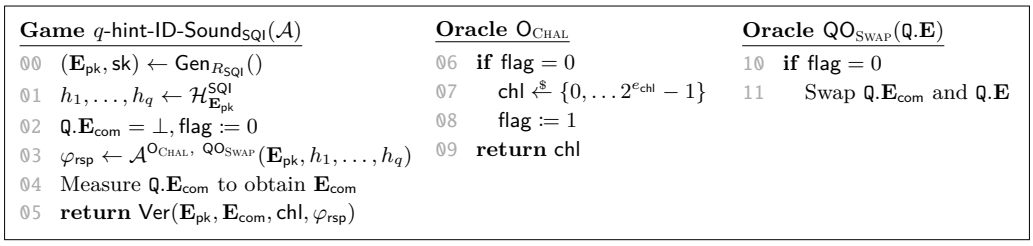

Fig. 8: The algebraic hint-ID-Sound game for SQIsign, defined relative to the number of hints q.

$\mathsf{QO}_{\mathrm{SWAP}}$. When the adversary calls the oracle with a commitment, the oracle stores it in the game state. To make this operation unitary, we implement it as a (controlled) swap. After the adversary receives the challenge, it can no longer change the commitment.

We show that if Σ has high commitment min-entropy and hint-wHVZK, then the EUF-CMA of SIG[Σ] reduces to hint-ID-Sound in the QROM. The proof is a straightforward application of results from [DFMS19, DFM20, GHHM21]. Hence, we defer it to Appendix C.1.

Lemma 5.9. *Let Σ be a Σ-protocol for a relation R and let $\mathcal{H}$ be a hint distribution. Let $\mathcal{A}$ be a (quantum)* EUF-CMA *adversary issuing at most q_s classical queries to the signing oracle O_{SIGN} and at most q_r quantum queries to the random oracle* RO*. Then there exists a (quantum) q_s-*hint-ID-Sound *adversary $\mathcal{B}$ and a q_s-*hint-wHVZK *adversary $\mathcal{D}$ such that*

$$\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{SIG}[\Sigma]}(\mathcal{A}) \leq (2q_r + 1)^2 \cdot \mathsf{Adv}^{q_s\text{-}\mathsf{hint\text{-}ID\text{-}Sound}}_{\Sigma,\mathcal{H}}(\mathcal{B}) + \mathsf{Adv}^{q_s\text{-}\mathsf{hint\text{-}wHVZK}}_{\Sigma,\mathcal{H}}(\mathcal{D}) + \frac{3q_s}{2}\sqrt{(q_s + q_r + 1) \cdot \mathsf{MinEnt}(\Sigma)}$$

and the running time of $\mathcal{B}$ and $\mathcal{D}$ is about that of $\mathcal{A}$.

We note that this reduction also works if the commitment in the hint-ID-Sound is required to be classical. We allow the commitment in the game to be quantum, so that for SQIsign we can lift this reduction to the AIM. We discuss this next.

5.3 Quantum security of SQIsign in the AIM

In this section, we analyze SQIsign in the AIM. We interpret SQIsign in our typed pseudocode framework as follows:

- The commitment: In SQIsign, the commitment is given by the j-invariant $j(E_{\mathsf{com}})$. In our model, it corresponds to a curve $\mathbf{E}_{\mathsf{com}}$
- The response: The response in SQIsign is an efficient representation of an isogeny $\varphi_{\mathsf{rsp}} : \mathbf{E}_{\mathsf{com}} \to \mathbf{E}_{\mathsf{chl}}$ of degree $< 2^{e_{\mathsf{rsp}}}$, where $\mathbf{E}_{\mathsf{chl}}$ is the challenge curve.
- Hash function: In the QROM, we model the hash function as a quantum oracle RO implementing a random function

$$\mathsf{RO} : \langle curve \rangle \times \langle curve \rangle \times \langle bits \rangle \to \langle bit \rangle^{e_{\mathsf{chl}}}.$$

  The signer obtains the challenge as $\mathsf{chl} = \mathsf{RO}(\mathbf{E}_{\mathsf{pk}}, \mathbf{E}_{\mathsf{com}}, \mathsf{msg})$.
- The starting curve: $\mathbf{E}_0$ is the curve of j-invariant of 1728 of known endomorphism ring. Thus, we can take $\mathbf{E}_{\mathsf{end}} \coloneqq \mathbf{E}_0$.

We present the algebraic game hint-ID-Sound for SQIsign in Fig. 8. The key observation is that if an algebraic adversary wins the game, then with overwhelming probability we can extract a non-trivial endomorphism of $\mathbf{E}_{\mathsf{pk}}$.

Theorem 5.10 (q-hint-OneEnd $\Rightarrow_{\mathsf{alg}}$ q-hint-ID-Sound$_{\mathsf{SQI}}$)**.** *Let $\mathcal{A}$ be a (quantum) algebraic adversary for q-hint-ID-Sound$_{\mathsf{SQI}}$. Then there is a (quantum) algebraic adversary $\mathcal{B}$ for q-hint-OneEnd such that*

$$\mathsf{Adv}^{q\text{-hint-ID-Sound}}_{\mathsf{SQI}}(\mathcal{A}) \leq \mathsf{Adv}^{q\text{-hint-OneEnd}}_{\mathcal{I}_{\mathcal{H}}}(\mathcal{B}) + \frac{1}{2^{e_{\mathsf{chl}}}} + \frac{1}{2\sqrt{p}},$$

and the running time of $\mathcal{B}$ is about that of $\mathcal{A}$.

Proof. On input a curve $\mathbf{E}$ and hints $h_1, \ldots, h_q$ in the q-hint-OneEnd-game, $\mathcal{B}$ simulates the game q-hint-ID-Sound for $\mathcal{A}$, using $\mathbf{E}$ as the public key curve. At the end of the game, $\mathcal{B}$ measures the commitment curve and explanation, and checks if $\mathcal{A}$ has won the game. The curves output by $\mathsf{Gen}_{R_{\mathsf{SQI}}}$ have statistical distance at most $\frac{1}{2}p^{-1/2}$ from the stationary distribution by Lemma 5.7. Hence, the advantage of $\mathcal{A}$ can decrease by at most $1/(2\sqrt{p})$ in this simulated game.

If $\mathcal{A}$ wins, let φ_{expl} be its explanation for $\mathbf{E}_{\mathsf{com}}$. The rest of the proof is divided into two cases, depending on the domain of φ_{expl}.

Case 1: $\varphi_{\mathsf{expl}} : \mathbf{E}_0 \to \mathbf{E}_{\mathsf{com}}$. Let $\psi = \hat{\varphi}_{\mathsf{chl}} \circ \varphi_{\mathsf{rsp}} \circ \varphi_{\mathsf{expl}} : \mathbf{E}_0 \to \mathbf{E}$. We know a basis $1, \beta_1, \beta_2, \beta_3$ for $\mathrm{End}(\mathbf{E}_0)$. Then $\psi \circ \beta_1 \circ \hat{\psi} \in \mathrm{End}(\mathbf{E}) \setminus \mathbb{Z}$.

Case 2: $\varphi_{\mathsf{expl}} : \mathbf{E} \to \mathbf{E}_{\mathsf{com}}$. Then $\psi = \hat{\varphi}_{\mathsf{expl}} \circ \hat{\varphi}_{\mathsf{rsp}} \circ \varphi_{\mathsf{chl}} \in \mathrm{End}(\mathbf{E})$. We will argue that with high probability $\psi \notin \mathbb{Z}$. Since $\deg(\varphi_{\mathsf{rsp}}) < 2^{e_{\mathsf{rsp}}}$ and $\deg(\varphi_{\mathsf{chl}}) = 2^e$, $\hat{\varphi}_{\mathsf{rsp}}$ can at most backtrack e_{rsp} steps of φ_{chl}. Write

$$\hat{\varphi}_{\mathsf{rsp}} \circ \varphi_{\mathsf{chl}} = \varphi' \circ [2^m] \circ \sigma \quad \text{and} \quad \varphi_{\mathsf{expl}} = \varphi'_{\mathsf{com}} \circ [2^{m_{\mathsf{com}}}] \circ \sigma_{\mathsf{com}},$$

where φ' and φ'_{com} have odd degrees, and σ and σ_{com} are cyclic with $\deg(\sigma) = 2^n$ and $\deg(\sigma_{\mathsf{com}}) = 2^{n_{\mathsf{com}}}$. We have $n \geq e - e_{\mathsf{rsp}} = e_{\mathsf{chl}}$.

Every challenge in SQIsign defines a different cyclic subgroup of order $2^{e_{\mathsf{chl}}}$ in $\mathbf{E}[2^{e_{\mathsf{chl}}}]$. Thus, $\ker\sigma \cap \mathbf{E}[2^{e_{\mathsf{chl}}}]$ is distributed uniformly randomly among the $2^{e_{\mathsf{chl}}}$ possible subgroups. For $\psi \in \mathbb{Z}$, we must have $\hat{\sigma}_{\mathsf{com}} \circ \sigma \in \mathbb{Z}$. Since the adversary committed to φ_{expl} before seeing the challenge, the probability that $\ker\sigma_{\mathsf{com}}$ contains the same cyclic subgroup is at most $2^{-e_{\mathsf{chl}}}$. □

The reduction from hint-ID-Sound to EUF-CMA in Lemma 5.9 can be lifted to the AIM, following the discussion in Remark 3.3. In this reduction, $\mathbf{E}_{\mathsf{com}}$ is obtained by measuring one of the quantum RO queries made by the EUF-CMA adversary. At this point, however, we do not wish to measure the *explanation* of $\mathbf{E}_{\mathsf{com}}$, as doing so could disturb the adversary's state. Instead, we model the commitment as a quantum curve in the hint-ID-Sound game, which allows us to postpone the measurement of the explanation until the end of the game.

Combining this reduction with the reduction from hint-EndRing to hint-OneEnd [PW24, ABD+25], we obtain our final security statement for SQIsign in the AIM + QROM. For completeness, we provide a proof in Appendix C.2.

Theorem 5.11. *Let $\mathcal{A}$ be a (quantum) algebraic* EUF-CMA *adversary for* SQIsign*, issuing at most q_s classical queries to the signing oracle O_{SIGN} and at most q_r quantum queries to the random oracle* RO*. Then there exist (quantum) algorithms $\mathcal{B}$ and $\mathcal{D}$ such that*

$$\begin{aligned}\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{SQI}}(\mathcal{A}) \leq\ & 4(q_r+1)^2 \cdot \mathsf{Adv}^{q_s\text{-hint-OneEnd}}(\mathcal{B}) + \mathsf{Adv}^{q_s\text{-hint-dist}}(\mathcal{D}) \\ & + \frac{4(q_r+1)^2}{2^{e_{\mathsf{chl}}}} + \frac{2(q_r+1)^2+q_s}{\sqrt{p}} + \frac{2q_s\sqrt{q_s+q_r+1}}{\sqrt[4]{p}}\end{aligned}$$

and the running time of $\mathcal{B}$ and $\mathcal{D}$ is about that of $\mathcal{A}$. Whenever $\mathcal{B}$ has advantage $\varepsilon \geq 2\log(p)/p$, there is a quantum algorithm $\mathcal{E}$ for t-hint-EndRing where

$$t = 96q_s/\varepsilon \quad \text{and} \quad \mathsf{Adv}^{t\text{-hint-EndRing}}(\mathcal{E}) \geq 1/4,$$

running in expected time $\mathsf{poly}(\log p)/\varepsilon$.

6 The DLOG to CDH reduction

In this section, we introduce our second main result in the AIM: the CDH problem and the DLOG problem are equivalent for all isogeny-based key exchanges. For CSIDH, a similar (quantum) result holds in the standard model [MZ22, GLM24], and it can also be shown (classically) in the $\mathsf{AGAM}^{\top}$ fairly straightforwardly [DHK+23, Thm. 7]. We thus focus on key exchanges based on SIDH, such as M-SIDH [FMP23] or terSIDH [BF23].

In the AIM, the DLOG to CDH reduction is significantly more intricate for mainly two reasons: 1. for all SIDH-derived key exchanges, the corresponding CDH and DLOG problems are not random self-reducible, and 2. an algebraic adversary in the AIM can produce any isogeny as an explanation. This means that the explanation does not need to be compatible with the protocol itself: for instance, while the secret isogenies in SIDH-derived key exchanges usually have a fixed degree, the explanation could instead have a different degree. Thus, extracting a valid DLOG solution is a priori hard. In our proof, we sidestep this issue by showing that, given a CDH oracle, it is always possible to solve the DLOG problem or to find a non-trivial endomorphism. Ultimately, we show that in the most general setting, the CDH problem and the DLOG problem are equivalent *assuming the hardness of the Endomorphism Ring problem*, whereas for all known instantiations of a SIDH-based key exchange the results hold unconditionally.

6.1 A generalized SIDH key exchange

In this section, we consider a generalization of SIDH key exchanges, which we call a *generalized SIDH key exchange*. The public parameters contain a prime p, a supersingular elliptic curve $\mathbf{E}_0$ defined over $\mathbb{F}_{p^2}$, and two lists $\mathfrak{A}$, $\mathfrak{B}$ of subgroups of $\mathbf{E}_0(\mathbb{F}_{p^{2k}})$ with $\mathfrak{A} \cap \mathfrak{B} = \emptyset$. These lists represent the additional information used in SIDH-based key exchanges to ensure isogenies commute; thus, they must satisfy the following properties:

(i) All lists have a succinct representation (polynomial in $\log p$): in most instances, these are described by two torsion points.

(ii) Given a curve $\mathbf{E}_0$, it is efficient to find lists $\mathfrak{A}, \mathfrak{B}$ in $\mathbf{E}_0$. We write $\mathfrak{A} \leftarrow \mathsf{GenList}_A(\mathbf{E}_0)$ to denote such an algorithm.

(iii) For any isogeny $\varphi_A : \mathbf{E}_0 \to \mathbf{E}_A$, given the list $\mathfrak{B} \subset \mathbf{E}_0$, an isogeny $\varphi_B : \mathbf{E}_0 \to \mathbf{E}_B$ that has kernel in $\mathfrak{B}$, and the list $\mathfrak{B}' \subset E_A$, where

$$\mathfrak{B}' = \varphi_A(\mathfrak{B}) = [\varphi_A(G) \mid G \in \mathfrak{B}],$$

it is efficient to compute the pushforward $\varphi'_B = [\varphi_A]_*\varphi_B$. We write $\varphi'_B = \mathsf{Pushforward}((\mathbf{E}_0, \mathfrak{B}), (\mathbf{E}_A, \mathfrak{B}'), \varphi_B)$. Note that the existence of a $\mathsf{Pushforward}$ algorithm implies we can also compute pullbacks: given $\mathfrak{B}$ and $\mathfrak{B}'$, as well as an isogeny φ'_B with kernel in $\mathfrak{B}'$, we can compute the pullback $[\varphi_A]^*\varphi'_B$ as $\mathsf{Pushforward}((\mathbf{E}_B, \mathfrak{B}'), (\mathbf{E}_0, \mathfrak{B}), \varphi'_B)$.

For ease of notation, given a list $\mathfrak{A}$, we call an isogeny with kernel in $\mathfrak{A}$ an $\mathfrak{A}$-isogeny. For the sake of presentation, we assume that all $\mathfrak{A}$- and $\mathfrak{B}$-isogenies are cyclic, that is, that the lists $\mathfrak{A}$ and $\mathfrak{B}$ contain only cyclic subgroups. However, all of our results can easily be extended to the handle isogenies that are not cyclic.

We are now ready to introduce our generalization. We say *generalized SIDH key exchange* is any key exchange of the following form:

- KeyGen_A: Alice samples a random $\mathfrak{A}$-isogeny: i.e., she samples a subgroup G_A in the set $\mathfrak{A}$ and computes the isogeny $\varphi_A : \mathbf{E}_0 \to \mathbf{E}_A$ with kernel G_A. The secret and public keys are

$$\mathsf{sk}_A = \varphi_A, \qquad \mathsf{pk}_A = (\mathbf{E}_A, \varphi_A(\mathfrak{B}))\,.$$

- KeyGen_B: Bob, similarly, samples a random $\mathfrak{B}$-isogeny: i.e., he samples a random subgroup G_B in the set $\mathfrak{B}$ and computes the isogeny $\varphi_B : \mathbf{E}_0 \to \mathbf{E}_B$ with kernel G_B. The secret and public keys are

$$\mathsf{sk}_B = \varphi_B, \qquad \mathsf{pk}_B = (\mathbf{E}_B, \varphi_B(\mathfrak{A}))\,.$$

Distribution $\mathcal{D}_1$	**Distribution $\mathcal{D}_2$**
00 $\mathbf{E}_0 \leftarrow \mathsf{SS}(p)$	04 $\mathbf{E}_X \leftarrow \mathsf{SS}(p)$
01 $\mathfrak{A} \leftarrow \mathsf{GenList}_X(\mathbf{E}_0)$	05 $\mathfrak{A}' \leftarrow \mathsf{GenList}_X(\mathbf{E}_X)$
02 Sample a $\mathfrak{A}$-isogeny $\varphi_X : \mathbf{E}_0 \to \mathbf{E}_X$	06 Sample a $\mathfrak{A}'$-isogeny $\widehat{\varphi}_X : \mathbf{E}_X \to \mathbf{E}_0$
03 **return** $(\mathbf{E}_0, \mathfrak{A}, \mathbf{E}_X, \varphi_X)$	07 $\mathfrak{A} \leftarrow \mathsf{GenList}_X(\mathbf{E}_0, \ker(\varphi_X))$
	08 **return** $(\mathbf{E}_0, \mathfrak{A}, \mathbf{E}_X, \varphi_X)$

Fig. 9: For $X \in \{A, B\}$, we require that the distributions are perfectly indistinguishable.

– $\mathsf{SharedKey}_A$: Given Bob's public key $(\mathbf{E}_B, \mathfrak{A}')$, Alice computes her shared secret $\mathbf{E}_{AB}$ as the codomain of the isogeny $\varphi'_A : \mathbf{E}_B \to \mathbf{E}_{AB}$, where

$$\varphi'_A = \mathsf{Pushforward}((\mathbf{E}_0, \mathfrak{A}), (\mathbf{E}_B, \mathfrak{A}'), \varphi_A).$$

– $\mathsf{SharedKey}_B$: Similarly, given Alice's public key $(\mathbf{E}_A, \mathfrak{B}')$, Bob computes his shared secret $\mathbf{E}_{BA}$ as the codomain of the isogeny $\varphi'_B : \mathbf{E}_A \to \mathbf{E}_{BA}$, where

$$\varphi'_B = \mathsf{Pushforward}((\mathbf{E}_0, \mathfrak{B}), (\mathbf{E}_A, \mathfrak{B}'), \varphi_B).$$

The public parameters are thus the prime p, the curve $\mathbf{E}_0$, as well as algorithms to sample lists $\mathfrak{A}, \mathfrak{B} \in \mathbf{E}_0$, to obtain an efficient representation of such lists, and to compute the $\mathsf{Pushforward}$ operation.

Example 6.1. In M-SIDH [FMP23], the lists $\mathfrak{A}$ and $\mathfrak{B}$ consist of cyclic subgroups of $\mathbf{E}_0(\mathbb{F}_{p^2})$ of order A and B respectively, where A and B are smooth and coprime. The lists have a succinct representation: for instance, $\mathfrak{A}$ is represented by a basis P_A, Q_A of $\mathbf{E}_0[A]$, and every subgroup of $\mathfrak{A}$ is of the form $\langle P_A + [\mathsf{sk}]Q_A \rangle$ with $\mathsf{sk} \in \mathbb{Z}/A\mathbb{Z}$. Likewise, $\varphi_B(\mathfrak{A})$ is represented by the basis $[\alpha]\varphi_B(P_A), [\alpha]\varphi_B(Q_A)$, for a secret constant $\alpha \in (\mathbb{Z}/A\mathbb{Z})^\times$. The algorithm $\mathsf{GenList}_A(\mathbf{E}_0)$ simply samples a random basis for $\mathbf{E}_0[A]$. The $\mathsf{Pushforward}$ algorithm is also efficient: given an $\mathfrak{A}$-isogeny φ_A with kernel generator $P_A + [\mathsf{sk}]Q_A$, the kernel of $[\varphi_B]_*\varphi_A$ is generated by $[\alpha]\varphi_B(P_A) + [\mathsf{sk}]([\alpha]\varphi_B Q_A)$.

For the security reduction, we require two additional properties:

(iv) Extraction: There is an efficiently computable function $\mathsf{Extract}$ that, given a curve $\mathbf{E}$ with a list $\mathfrak{A}$, an isogeny $\varphi : \mathbf{E} \to \mathbf{F}$ and a target curve $\mathbf{E}_{\mathsf{target}}$, checks whether the isogeny is of the form $\varphi = \varphi' \circ \varphi_A$, where $\varphi_A : \mathbf{E} \to \mathbf{E}_{\mathsf{target}}$ is an $\mathfrak{A}$-isogeny and $\varphi' : \mathbf{E}_{\mathsf{target}} \to \mathbf{F}$. If so, it returns φ_A, otherwise, it returns $\perp$.

(v) Sampling a list with a subgroup: Given a curve $\mathbf{E}$ and a subgroup G, there is an efficient algorithm that samples a list containing G (or outputs $\perp$). We write $\mathfrak{A} \leftarrow \mathsf{GenList}_A(\mathbf{E}, G)$. We require that the two distributions in Fig. 9 are perfectly indistinguishable.

These conditions are very mild: all known instantiations of a generalized SIDH key exchange can be made to satisfy them. Property (v) can be generalized to the case where the distributions are statistically or computationally indistinguishable. This would incur an additive loss in our reduction equal to the distinguishing advantage of the adversary.

Example 6.2. Let us continue Example 6.1 for M-SIDH. First, we consider extraction. Given an isogeny $\varphi : \mathbf{E} \to \mathbf{F}$, we first divide it by every prime factor of A until it is no longer divisible, obtaining an isogeny ψ. If A does not divide $\deg \psi$, we output $\perp$. Otherwise, the group

$$G = \ker \psi \cap \mathbf{E}[A] = \widehat{\psi}(\mathbf{F}[A])$$

is a cyclic subgroup of order A. Let R_A be a generator of G, and write $R_A = [x]P_A + [y]Q_A$. If x is not invertible, we output $\perp$. Otherwise, $G = \langle P_A + [x^{-1}y]Q_A \rangle \in \mathfrak{A}$. Let φ_A be the $\mathfrak{A}$-isogeny with kernel G. If the codomain of φ_A is $\mathbf{E}_{\mathsf{target}}$ we output φ_A, else we output $\perp$.

Next, we define $\mathsf{GenList}_A(\mathbf{E}, G)$. If G is not a cyclic subgroup of $\mathbf{E}$ of order A, it outputs $\perp$. Otherwise, we sample a random basis P_A, Q_A for $\mathbf{E}[A]$. If G is of the form $\langle P_A + [x]Q_A \rangle$ we output (P_A, Q_A), else we output (Q_A, P_A). This definition ensures that the two distributions in Fig. 9 are perfectly indistinguishable. When a curve $\mathbf{E}$ is sampled from the stationary distribution and a random A-isogeny $\varphi_A : \mathbf{E} \to \mathbf{F}$ is computed, the resulting curve $\mathbf{F}$ is also distributed according to the stationary distribution. Consequently, $\mathcal{D}_1$ and $\mathcal{D}_2$ induce the same distribution on $(\mathbf{E}_0, \mathbf{E}_A, \varphi_A)$. Conditioned on this tuple, both distributions output a list $\mathfrak{A}$ represented by a random basis of $\mathbf{E}_0[A]$ such that $\ker(\varphi_A) \in \mathfrak{A}$.

6.2 Problem statements

Using this framework, we now define the DLOG and CDH problems for generalized SIDH key exchanges. We work with a variable-base version of both CDH and DLOG, i.e. the starting curve $\mathbf{E}_0$ is not fixed, and we thus expect a CDH/DLOG solver to be able to solve instances with different starting curves. This is, in some sense, the most general case, and it allows us to obtain the strongest results in the DLOG-to-CDH reduction. We discuss the fixed-basis case in Appendix E.

Problem 6.3 (CDH for generalized SIDH key exchanges). Let $\mathbf{E}_0 \leftarrow \mathsf{SS}(p)$ be a supersingular elliptic curve sampled from the stationary distribution with two lists $\mathfrak{A} \leftarrow \mathsf{GenList}_A(\mathbf{E}_0)$ and $\mathfrak{B} \leftarrow \mathsf{GenList}_B(\mathbf{E}_0)$. Let $\varphi_A : \mathbf{E}_0 \to \mathbf{E}_A$ and $\varphi_B : \mathbf{E}_0 \to \mathbf{E}_B$ be an $\mathfrak{A}$-isogeny and a $\mathfrak{B}$-isogeny, respectively.
Given the public parameters $(\mathbf{E}_0, \mathfrak{A}, \mathfrak{B})$, as well as $(\mathbf{E}_A, \varphi_A(\mathfrak{B}))$ and $(\mathbf{E}_B, \varphi_B(\mathfrak{A}))$, compute the curve $\mathbf{E}_{AB}$ that is the codomain of $[\varphi_A]_*(\varphi_B) = [\varphi_B]_*(\varphi_A)$.

We call a tuple consisting of $(\mathbf{E}_0, \mathfrak{A}, \mathfrak{B})$, $(\mathbf{E}_A, \varphi_A(\mathfrak{B}))$, $(\mathbf{E}_B, \varphi_B(\mathfrak{A}))$, and the corresponding CDH solution $\mathbf{E}_{AB}$ a *CDH quadruple*.

Note that, unlike the case of CSIDH, the construction is not symmetric (i.e., the two parties sample their isogenies from distinct lists): the DLOG problem thus consists of two instances (with the same starting curve $\mathbf{E}_0$) and asks to find either one of the two isogenies.

Problem 6.4 (DLOG for generalized SIDH key exchanges). Let $\mathbf{E}_0 \leftarrow \mathsf{SS}(p)$ be a supersingular elliptic curve sampled from the stationary distribution with two lists $\mathfrak{A} \leftarrow \mathsf{GenList}_A(\mathbf{E}_0)$ and $\mathfrak{B} \leftarrow \mathsf{GenList}_B(\mathbf{E}_0)$. Let $\varphi_A : \mathbf{E}_0 \to \mathbf{E}_A$ and $\varphi_B : \mathbf{E}_0 \to \mathbf{E}_B$ be an $\mathfrak{A}$-isogeny and a $\mathfrak{B}$-isogeny, respectively.
Given the public parameters $(\mathbf{E}_0, \mathfrak{A}, \mathfrak{B})$, as well as $(\mathbf{E}_A, \varphi_A(\mathfrak{B}))$ and $(\mathbf{E}_B, \varphi_B(\mathfrak{A}))$, compute either:

- an isogeny $\varphi_A^* : \mathbf{E}_0 \to \mathbf{E}_A$ such that $\varphi_A^*(\mathfrak{B}) = \varphi_A(\mathfrak{B})$,[7]
- an isogeny $\varphi_B^* : \mathbf{E}_0 \to \mathbf{E}_B$ such that $\varphi_B^*(\mathfrak{A}) = \varphi_B(\mathfrak{A})$.

We also define a weaker version of the DLOG problem, where the adversary also wins if it recovers one non-trivial endomorphism of $\mathbf{E}_A$ or $\mathbf{E}_B$.

Problem 6.5 (weakDLOG for generalized SIDH key exchanges). Let $\mathbf{E}_0 \leftarrow \mathsf{SS}(p)$ be a supersingular elliptic curve sampled from the stationary distribution with two lists $\mathfrak{A} \leftarrow \mathsf{GenList}_A(\mathbf{E}_0)$ and $\mathfrak{B} \leftarrow \mathsf{GenList}_B(\mathbf{E}_0)$. Let $\varphi_A : \mathbf{E}_0 \to \mathbf{E}_A$ and $\varphi_B : \mathbf{E}_0 \to \mathbf{E}_B$ be an $\mathfrak{A}$-isogeny and a $\mathfrak{B}$-isogeny, respectively.
Given the public parameters $(\mathbf{E}_0, \mathfrak{A}, \mathfrak{B})$, as well as $(\mathbf{E}_A, \varphi_A(\mathfrak{B}))$ and $(\mathbf{E}_B, \varphi_B(\mathfrak{A}))$, compute one of the following:

- an isogeny $\varphi_A^* : \mathbf{E}_0 \to \mathbf{E}_A$ such that $\varphi_A^*(\mathfrak{B}) = \varphi_A(\mathfrak{B})$,
- an isogeny $\varphi_B^* : \mathbf{E}_0 \to \mathbf{E}_B$ such that $\varphi_B^*(\mathfrak{A}) = \varphi_B(\mathfrak{A})$,

[7] In most instantiations, the isogeny φ_A is uniquely determined by its action on the $\mathfrak{B}$ set, so $\varphi_A^* = \varphi_A$ (and similarly for φ_B). But it may be possible to imagine instantiations where this is not necessarily the case, so we allow the adversary to produce any "valid" DLOG answer.

– one non-trivial endomorphism in $\mathrm{End}(\mathbf{E}_A)$,

– one non-trivial endomorphism in $\mathrm{End}(\mathbf{E}_B)$.

Definition 6.6. *We write* $\mathsf{Adv}^{\mathsf{CDH}}$, $\mathsf{Adv}^{\mathsf{DLOG}}$, $\mathsf{Adv}^{\mathsf{weakDLOG}}$ *for the advantage of the adversary in solving the CDH problem, the DLOG problem, and the weakDLOG problem for a generalized SIDH key exchange.*

6.3 The reduction

We finally obtain our main result.

Theorem 6.7. *For any algebraic adversary* $\mathcal{A}$ *that solves the CDH problem for generalized SIDH key exchanges, there exists an algebraic algorithm* $\mathcal{B}$ *such that*

$$\mathsf{Adv}^{\mathsf{CDH}}(\mathcal{A}) \leq 7 \cdot \mathsf{Adv}^{\mathsf{weakDLOG}}(\mathcal{B})$$

and the running time of $\mathcal{B}$ *is about that of* $\mathcal{A}$*.*

Proof. The adversary $\mathcal{A}$ against CDH, given $\big((\mathbf{E}_0, \mathfrak{A}, \mathfrak{B}), (\mathbf{E}_A, \mathfrak{B}'), (\mathbf{E}_B, \mathfrak{A}')\big)$, produces a fourth curve $\mathbf{E}_{AB}$. Since the adversary is algebraic, it must explain its outputs $\mathbf{E}_{AB}$ with one of four possible explanations:

1. An isogeny $\varphi_{\mathsf{expl}} : \mathbf{E}_0 \to \mathbf{E}_{AB}$, or
2. An isogeny $\varphi_{\mathsf{expl}} : \mathbf{E}_A \to \mathbf{E}_{AB}$,
3. An isogeny $\varphi_{\mathsf{expl}} : \mathbf{E}_B \to \mathbf{E}_{AB}$,
4. An isogeny $\varphi_{\mathsf{expl}} : \mathbf{E}_{\mathsf{end}} \to \mathbf{E}_{AB}$, where $\mathrm{End}(\mathbf{E}_{\mathsf{end}})$ is known.

We construct a weakDLOG adversary $\mathcal{B}$ (Fig. 10): it receives a weak DLOG challenge $\big((\mathbf{E}_0, \mathfrak{A}, \mathfrak{B}), (\mathbf{E}_A, \mathfrak{B}'), (\mathbf{E}_B, \mathfrak{A}')\big)$ and has to produce either an $\mathfrak{A}$-isogeny from $\mathbf{E}_0$ to $\mathbf{E}_A$, a $\mathfrak{B}$-isogeny from $\mathbf{E}_0$ to $\mathbf{E}_B$, or an endomorphism of $\mathbf{E}_A$ or $\mathbf{E}_B$.

First, $\mathcal{B}$ makes two guesses: 1. which explanation the adversary $\mathcal{A}$ will produce, and 2. whether our strategy for computing a DLOG solution from the explanation will succeed. If either guess is incorrect, $\mathcal{B}$ aborts. We handle each case separately:

– The adversary $\mathcal{A}$ will explain $\mathbf{E}_{AB}$ by $\varphi_{\mathsf{expl}} : \mathbf{E}_A \to \mathbf{E}_{AB}$.

- If the adversary will produce an isogeny from which we can extract a DLOG solution, the algorithm $\mathcal{B}$ simply passes the input $\big((E_0, \mathfrak{A}, \mathfrak{B}), (\mathbf{E}_A, \mathfrak{B}'), (\mathbf{E}_B, \mathfrak{A}')\big)$ to $\mathcal{A}$. When $\mathcal{A}$ responds with $\mathbf{E}_{AB}$, together with an algebraic explanation $\varphi_{\mathsf{expl}} : \mathbf{E}_A \to \mathbf{E}_{AB}$, the algorithm $\mathcal{B}$ extracts the $\mathfrak{B}$-isogeny $\varphi'_B \leftarrow \mathsf{Extract}_{\mathfrak{B}}(\varphi_{\mathsf{expl}}, \mathbf{E}_{AB})$. Then, $\mathcal{B}$ maps φ'_B back to $\mathbf{E}_0$ to obtain $\varphi_B \leftarrow \mathsf{Pushforward}((\mathbf{E}_A, \mathfrak{B}'), (\mathbf{E}_0, \mathfrak{B}), \varphi'_B)$, which it outputs as the DLOG solution.[8]
- If the adversary will produce an isogeny from which *we cannot* extract a DLOG solution, i.e. such that the previous case would fail, we proceed differently. The algorithm $\mathcal{B}$ samples a random $\mathfrak{B}$-isogeny $\psi_B : \mathbf{E}_0 \to \mathbf{F}_B$,[9] computes $\psi'_B : \mathbf{E}_A \to \mathbf{F}_{AB}$ as $\mathsf{Pushforward}((\mathbf{E}_0, \mathfrak{B}), (\mathbf{E}_A, \mathfrak{B}'), \psi_B)$, and computes the CDH challenge

$$\big((\mathbf{E}_0, \mathfrak{A}, \mathfrak{B}),\ (\mathbf{E}_A, \mathfrak{B}'),\ (\mathbf{F}_B, \psi_B(\mathfrak{A}))\big).$$

[8] If $\mathsf{Extract}_{\mathfrak{B}}(\varphi_{\mathsf{expl}}, \mathbf{E}_{AB}) \neq \bot$, the isogeny φ_B is guaranteed to be a $\mathfrak{B}$-isogeny from $\mathbf{E}_0$ to $\mathbf{E}_B$. However, a priori, it may still not be a valid DLOG response if it does not map $\mathfrak{A}$ to $\mathfrak{A}'$. This is not a problem: if φ_B is not a valid DLOG response, it means that $\mathcal{B}$ had guessed incorrectly.

[9] Throughout the reduction, the curves $\mathbf{E}_0$, $\mathbf{E}_A$, and $\mathbf{E}_B$ denote the input curves and $\mathbf{E}_{AB}$ is the CDH solution to the input. The curves that are generated as part of the reduction are denoted by $\mathbf{F}_\bullet$. Similarly, isogenies $\varphi_\bullet$ are those used in the generation of the input, whereas those computed as part of the reduction are $\psi_\bullet$ and $\sigma_\bullet$.

Adversary $\mathcal{B}$

Input: A weak DLOG challenge $\mathsf{chl} = \big((\mathbf{E}_0, \mathfrak{A}, \mathfrak{B}), (\mathbf{E}_A, \mathfrak{B}'), (\mathbf{E}_B, \mathfrak{A}')\big)$
Output: A weak DLOG solution
Subroutine: An adversary $\mathcal{A}$ solving CDH

00 Sample $\mathsf{expl} \xleftarrow{\$} \{0, A, B, \mathsf{end}\}$ \\ this guesses the type of explanation
01 Sample $\mathsf{case} \xleftarrow{\$} \{\mathsf{good}, \mathsf{bad}\}$ \\ this guesses if the explanation is compatible
02 **if** $\mathsf{expl} = A$
03 **if** $\mathsf{case} = \mathsf{good}$
04 $(\mathbf{E}_{AB}, \varphi_{\mathsf{expl}}) \leftarrow \mathcal{A}(\mathsf{chl})$
05 **if** $\mathrm{domain}(\varphi_{\mathsf{expl}}) \neq \mathbf{E}_A$ **then abort**
06 $\varphi_{\mathsf{good}} \leftarrow \mathsf{Extract}_{\mathfrak{B}}(\varphi_{\mathsf{expl}}, \mathbf{E}_{AB})$
07 **return** $\mathsf{Pushforward}((\mathbf{E}_A, \mathfrak{B}'), (\mathbf{E}_0, \mathfrak{B}), \varphi_{\mathsf{good}})$ \\ return φ_B
08 **elseif** $\mathsf{case} = \mathsf{bad}$
09 Sample a $\mathfrak{B}$-isogeny $\psi_B : \mathbf{E}_0 \to \mathbf{F}_B$
10 Compute $\psi'_B : \mathbf{E}_A \to \mathbf{F}_{AB}$ as $\mathsf{Pushforward}((\mathbf{E}_0, \mathfrak{B}), (\mathbf{E}_A, \mathfrak{B}'), \psi_B)$
11 $(\mathbf{F}_{AB}, \varphi_{\mathsf{expl}}) \leftarrow \mathcal{A}\big((\mathbf{E}_0, \mathfrak{A}, \mathfrak{B}), (\mathbf{E}_A, \mathfrak{B}'), (\mathbf{F}_B, \psi_B(\mathfrak{A}))\big)$
12 **if** $\mathrm{domain}(\varphi_{\mathsf{expl}}) \neq \mathbf{E}_A$ **then abort**
13 $\sigma_B \leftarrow \mathsf{Extract}_{\mathfrak{B}}(\varphi_{\mathsf{expl}}, \mathbf{F}_{AB})$
14 **if** $\sigma_B = \perp$ **return** $\widehat{\varphi}_{\mathsf{expl}} \circ \psi'_B$ **else return** $\widehat{\sigma}_B \circ \psi'_B$ \\ return an endomorphism of $\mathbf{E}_A$
15 **elseif** $\mathsf{expl} = B$
16 **if** $\mathsf{case} = \mathsf{good}$
17 $(\mathbf{E}_{AB}, \varphi_{\mathsf{expl}}) \leftarrow \mathcal{A}(\mathsf{chl})$
18 $\varphi_{\mathsf{good}} \leftarrow \mathsf{Extract}_{\mathfrak{A}}(\varphi_{\mathsf{expl}}, \mathbf{E}_{AB})$
19 **if** $\mathrm{domain}(\varphi_{\mathsf{expl}}) \neq \mathbf{E}_B$ **or** $\varphi_{\mathsf{good}} = \perp$ **then abort**
20 **return** $\mathsf{Pushforward}((\mathbf{E}_B, \mathfrak{A}'), (\mathbf{E}_0, \mathfrak{A}), \varphi_{\mathsf{good}})$ \\ return φ_A
21 **elseif** $\mathsf{case} = \mathsf{bad}$
22 Sample an $\mathfrak{A}$-isogeny $\psi_A : \mathbf{E}_0 \to \mathbf{F}_A$
23 Compute $\psi'_A : \mathbf{E}_B \to \mathbf{F}_{AB}$ as $\mathsf{Pushforward}((\mathbf{E}_0, \mathfrak{B}), (\mathbf{E}_B, \mathfrak{A}'), \psi_A)$
24 $(\mathbf{F}_{AB}, \varphi_{\mathsf{expl}}) \leftarrow \mathcal{A}\big((\mathbf{E}_0, \mathfrak{A}, \mathfrak{B}), (\mathbf{F}_A, \psi_A(\mathfrak{B})), (\mathbf{E}_B, \mathfrak{A}')\big)$
25 **if** $\mathrm{domain}(\varphi_{\mathsf{expl}}) \neq \mathbf{E}_B$ **then abort**
26 $\sigma_A \leftarrow \mathsf{Extract}_{\mathfrak{A}}(\varphi_{\mathsf{expl}}, \mathbf{F}_{AB})$
27 **if** $\sigma_A = \perp$ **return** $\widehat{\varphi_{\mathsf{expl}}} \circ \psi'_A$ **else return** $\widehat{\sigma}_A \circ \psi'_A$ \\ return an endomorphism of $\mathbf{E}_B$
28 **elseif** $\mathsf{expl} = 0$
29 **if** $\mathsf{case} = \mathsf{good}$
30 $(\mathbf{E}_{AB}, \varphi_{\mathsf{expl}}) \leftarrow \mathcal{A}(\mathsf{chl})$
31 **if** $\mathrm{domain}(\varphi_{\mathsf{expl}}) \neq \mathbf{E}_0$ **then abort**
32 $\varphi_{\mathsf{good}} \leftarrow \mathsf{Extract}_{\mathfrak{A}}(\varphi_{\mathsf{expl}}, \mathbf{E}_A)$
33 **return** φ_{good} \\ return φ_A
34 **if** $\mathsf{case} = \mathsf{bad}$
35 Sample lists $\mathfrak{A}' \leftarrow \mathsf{GenList}_A(\mathbf{E}_A)$ and $\mathfrak{B}' \leftarrow \mathsf{GenList}_B(\mathbf{E}_A)$
36 Sample an $\mathfrak{A}'$-isogeny $\psi_A : \mathbf{E}_A \to \mathbf{F}_A$ and a $\mathfrak{B}'$-isogeny $\psi_B : \mathbf{E}_A \to \mathbf{F}_B$
37 Compute $\psi'_A = [\psi_B]_* \psi_A : \mathbf{F}_B \to \mathbf{F}_{AB}$ and $\psi'_B = [\psi_A]_* \psi_B : \mathbf{F}_A \to \mathbf{F}_{AB}$
38 $\mathsf{chl}' \leftarrow ((\mathbf{E}_A, \mathfrak{A}', \mathfrak{B}'), (\mathbf{F}_A, \psi_B(\mathfrak{A}')), (\mathbf{F}_B, \psi_A(\mathfrak{B}')))$
39 $(\mathbf{F}_{AB}, \varphi_{\mathsf{expl}}) \leftarrow \mathcal{A}(\mathsf{chl}')$
40 **if** $\mathrm{domain}(\varphi_{\mathsf{expl}}) \neq \mathbf{E}_A$ **then abort**
41 $\sigma_A \leftarrow \mathsf{Extract}_{\mathfrak{A}'}(\varphi_{\mathsf{expl}}, \mathbf{F}_A)$
42 **if** $\sigma_A = \perp$ **return** $\widehat{\varphi}_{\mathsf{expl}} \circ \psi'_B \circ \psi_A$ **else return** $\widehat{\sigma}_A \circ \psi_A$ \\ return an endomorphism of $\mathbf{E}_A$
43 **elseif** $\mathsf{expl} = \mathsf{end}$
44 Sample a $\mathfrak{B}$-isogeny $\psi_B : \mathbf{E}_0 \to \mathbf{F}_B$
45 Compute $\psi'_B : \mathbf{E}_A \to \mathbf{F}_{AB}$ as $\mathsf{Pushforward}((\mathbf{E}_0, \mathfrak{B}), (\mathbf{E}_A, \mathfrak{B}'), \psi_B)$
46 $(\mathbf{F}_{AB}, \varphi_{\mathsf{expl}}) \leftarrow \mathcal{A}\big((\mathbf{E}_0, \mathfrak{A}, \mathfrak{B}), (\mathbf{E}_A, \mathfrak{B}'), (\mathbf{F}_B, \psi_B(\mathfrak{A}))\big)$
47 **if** $\mathrm{domain}(\varphi_{\mathsf{expl}}) \neq \mathbf{E}_{\mathsf{end}}$ **then abort**
48 Let β be a non-scalar endomorphism of $\mathbf{E}_{\mathsf{end}}$
49 **return** $\widehat{\psi'_B} \circ \varphi_{\mathsf{expl}} \circ \beta \circ \widehat{\varphi}_{\mathsf{expl}} \circ \psi'_B$ \\ return an endomorphism of $\mathbf{E}_A$

Fig. 10: Adversary $\mathcal{B}$ for the proof of Theorem 6.7.

If the adversary $\mathcal{A}$ solves the CDH challenge, it responds with $\mathbf{F}_{AB}$, together with an explanation $\varphi_{\mathsf{expl}} : \mathbf{E}_A \to \mathbf{F}_{AB}$. Our goal is to find an isogeny $\tau : \mathbf{E}_A \to \mathbf{F}_{AB}$ such that $\hat{\tau} \circ \psi'_B$ is a non-scalar endomorphism of $\mathbf{E}_A$. Observe that the endomorphism is scalar if and only if $\tau = [m] \circ \psi'_B$ where m is an integer.
Let $\sigma_B = \mathsf{Extract}_{\mathfrak{B}}(\varphi_{\mathsf{expl}}, \mathbf{F}_{AB})$. If $\sigma_B = \bot$, then ψ'_B cannot factor through φ_{expl}, so we can pick $\tau = \varphi_{\mathsf{expl}}$. Else, if $\sigma_B \neq \bot$, it could be that $\varphi_{\mathsf{expl}} = [m] \circ \psi'_B$. However, by the assumed case, we could not have that $\sigma_B = \pm\psi'_B$. Hence, we can pick $\tau = \sigma_B$ instead.

– The adversary $\mathcal{A}$ will explain $\mathbf{E}_{AB}$ by $\varphi_{\mathsf{expl}} : \mathbf{E}_B \to \mathbf{E}_{AB}$. This case proceeds similarly to the previous one, by swapping the roles of $\mathbf{E}_A$ and $\mathbf{E}_B$, φ_A and φ_B, and $\mathfrak{A}$ and $\mathfrak{B}$.

– The adversary $\mathcal{A}$ will explain $\mathbf{E}_{AB}$ by $\varphi_{\mathsf{expl}} : \mathbf{E}_0 \to \mathbf{E}_{AB}$.
  • If the adversary will produce an isogeny from which we can extract a DLOG solution, $\mathcal{B}$ computes it by extracting an $\mathfrak{A}$-isogeny from φ_{expl}.
  • Otherwise, $\mathcal{B}$ samples a new challenge using $\mathbf{E}_A$ as the starting curve:

$$((\mathbf{E}_A, \mathfrak{A}', \mathfrak{B}'), (\mathbf{F}_A, \psi_B(\mathfrak{A}'), (\mathbf{F}_B, \psi_A(\mathfrak{B}'))).$$

Since the distributions in Fig. 9 are perfectly indistinguishable, $\mathbf{E}_A$ has the same distribution as $\mathbf{E}_0$, so the new challenge is identically distributed to a CDH challenge. If $\mathcal{A}$ solves the challenge, it responds with $\mathbf{F}_{AB}$, together with an explanation $\varphi_{\mathsf{expl}} : \mathbf{E}_A \to \mathbf{F}_{AB}$. From this, we compute a non-scalar endomorphism of $\mathbf{E}_A$.
Let $\sigma_A \leftarrow \mathsf{Extract}_{\mathcal{A}'}(\varphi_{\mathsf{expl}}, \mathbf{F}_A)$. If $\sigma_A = \bot$, ψ_A does not divide φ_{expl}, so we output $\widehat{\varphi}_{\mathsf{expl}} \circ \psi'_B \circ \psi_A$. Else, if $\sigma_A \neq \bot$, we know that $\sigma_A \neq \pm\psi_A$, so we output $\widehat{\sigma}_A \circ \psi_A$.

– The adversary $\mathcal{A}$ will explain $\mathbf{E}_{AB}$ by $\varphi_{\mathsf{expl}} : \mathbf{E}_{\mathsf{end}} \to \mathbf{E}_{AB}$. In this case, the algorithm $\mathcal{B}$ creates a CDH challenge in the same way as the bad case for $\mathbf{E}_A$. Given an explanation $\varphi_{\mathsf{expl}} : \mathbf{E}_{\mathsf{end}} \to \mathbf{F}_{AB}$, we have an isogeny from $\mathbf{E}_{\mathsf{end}}$ to $\mathbf{E}_A$, $\widehat{\psi'_B} \circ \varphi_{\mathsf{expl}}$. Since we know the endomorphism ring of $\mathbf{E}_{\mathsf{end}}$, we can compute the "lollipop" of any non-scalar endomorphism β of $\mathbf{E}_{\mathsf{end}}$, to obtain a non-scalar endomorphism of $\mathbf{E}_A$.

Note that if $\mathcal{B}$ guesses the correct case (both the domain of the explanation isogeny, and whether the explanation isogeny contains a valid DLOG solution), the adversary $\mathcal{B}$ wins the weak DLOG problem with the same advantage that $\mathcal{A}$ has in solving the CDH problem. Since $\mathcal{B}$'s guessing is independent of $\mathcal{A}$'s view, the security loss is a constant factor 7 (the case $\mathsf{expl} = \mathsf{end}$ is not affected by whether $\mathsf{case} = \mathsf{good}$ or $\mathsf{case} = \mathsf{bad}$). □

Theorem 6.8. *For any algebraic adversary $\mathcal{A}$ that solves the weak DLOG problem for generalized SIDH key exchanges, there exist algebraic algorithms $\mathcal{B}$ and $\mathcal{C}$ such that*

$$\mathsf{Adv}^{\mathsf{weakDLOG}}(\mathcal{A}) \leq 2 \cdot \left(\mathsf{Adv}^{\mathsf{DLOG}}(\mathcal{B}) + \mathsf{Adv}^{\mathsf{EndRing}}(\mathcal{C})\right).$$

Proof. The weakDLOG adversary $\mathcal{A}$ outputs either a DLOG isogeny or an endomorphism. In the first case, it is straightforward that its output is a valid DLOG solution. In the second case, we need to show that a weakDLOG adversary that returns an endomorphism can be used to obtain the full endomorphism ring of a curve (sampled from the stationary distribution). To use the EndRing to OneEnd reduction by Page and Wesolowski [PW24], we need to embed a OneEnd query $\mathbf{E}$ (sampled from the stationary distribution) into a weakDLOG query, either as $\mathbf{E}_A$ or as $\mathbf{E}_B$.

To embed the OneEnd query $\mathbf{E}$ as $\mathbf{E}_A$, we proceed as in Fig. 9. We first generate a temporary list $\mathfrak{A}' \leftarrow \mathsf{GenList}_A(\mathbf{E})$ and sample a random $\mathfrak{A}'$-isogeny $\widehat{\varphi_A} : \mathbf{E} \to \mathbf{E}_0$. We then set $\mathfrak{A} \leftarrow \mathsf{GenList}_A(\mathbf{E}_0, \ker(\varphi_A))$. By Property (v), the tuple $(\mathbf{E}_0, \mathfrak{A}, \mathbf{E}, \varphi_A)$ is distributed correctly. Discarding the list $\mathfrak{A}'$, we generate the rest of the CDH challenge in the standard way, to obtain

$$((\mathbf{E}_0, \mathfrak{A}, \mathfrak{B}), (\mathbf{E}, \varphi_A(\mathfrak{B})), (\mathbf{E}_B, \varphi_B(\mathfrak{A})).$$

The loss factor 2 in the theorem statement comes from the choice of $\mathbf{E}_A$ or $\mathbf{E}_B$ when embedding the challenge. □

This shows that, assuming the EndRing problem is hard, the DLOG and CDH problems for generalized SIDH key exchanges are equivalent in the AIM. Furthermore, we can remove the EndRing hardness assumption for all known secure instantiations of a generalized SIDH key exchange, i.e., M-SIDH [FMP23], MD-SIDH [FMP23], binSIDH [BF23], binSIDH-hyb [BF23], terSIDH [BF23], terSIDH-hyb [BF23]. That is because all known instantiations use isogenies that are significantly shorter than p: hence, we could obtain a DLOG solution from the endomorphism ring of $\mathbf{E}_0$ and of a public key $\mathbf{E}_A$ (or, equivalently, $\mathbf{E}_B$) with overwhelming probability [BKM+24].

Acknowledgements

We would like to thank Luca De Feo, Jonas Meers and Ryan Rueger for valuable discussions about the project. This work was funded by the Danish Independent Research Council under project number 1026-00350B (RENAIS) and by the Swiss SNF Consolidator Grant No. 213766 (CryptonIs).

References

AAA+25. Marius A. Aardal, Gora Adj, Diego F. Aranha, Andrea Basso, Isaac Andrés Canales Martínez, Jorge Chávez-Saab, Maria Corte-Real Santos, Pierrick Dartois, Luca De Feo, Max Duparc, Jonathan Komada Eriksen, Tako Boris Fouotsa, Décio Luiz Gazzoni Filho, Basil Hess, David Kohel, Antonin Leroux, Patrick Longa, Luciano Maino, Michael Meyer, Kohei Nakagawa, Hiroshi Onuki, Lorenz Panny, Sikhar Patranabis, Christophe Petit, Giacomo Pope, Krijn Reijnders, Damien Robert, Francisco Rodríguez Henríquez, Sina Schaeffler, and Benjamin Wesolowski. SQIsign. Technical report, National Institute of Standards and Technology, 2025. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures.

ABD+25. Marius A. Aardal, Andrea Basso, Luca De Feo, Sikhar Patranabis, and Benjamin Wesolowski. A complete security proof of SQIsign. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 190–222. Springer, Cham, August 2025. doi:10.1007/978-3-032-01887-8_7.

ACL+22. Sarah Arpin, Mingjie Chen, Kristin E. Lauter, Renate Scheidler, Katherine E. Stange, and Ha T. N. Tran. Orientations and cycles in supersingular isogeny graphs. *CoRR*, abs/2205.03976, 2022.

ADMP20. Navid Alamati, Luca De Feo, Hart Montgomery, and Sikhar Patranabis. Cryptographic group actions and applications. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part II*, volume 12492 of *LNCS*, pages 411–439. Springer, Cham, December 2020. doi:10.1007/978-3-030-64834-3_14.

BBC+25. Andrea Basso, Giacomo Borin, Wouter Castryck, Maria Corte-Real Santos, Riccardo Invernizzi, Antonin Leroux, Luciano Maino, Frederik Vercauteren, and Benjamin Wesolowski. PRISM: Simple and compact identification and signatures from large prime degree isogenies. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025, Part III*, volume 15676 of *LNCS*, pages 300–332. Springer, Cham, May 2025. doi:10.1007/978-3-031-91826-1_10.

BBD+22. Jeremy Booher, Ross Bowden, Javad Doliskani, Tako Boris Fouotsa, Steven D. Galbraith, Sabrina Kunzweiler, Simon-Philipp Merz, Christophe Petit, Benjamin Smith, Katherine E. Stange, Yan Bo Ti, Christelle Vincent, José Felipe Voloch, Charlotte Weitkämper, and Lukas Zobernig. Failing to hash into supersingular isogeny graphs. Cryptology ePrint Archive, Report 2022/518, 2022. URL: https://eprint.iacr.org/2022/518.

BBG05. Dan Boneh, Xavier Boyen, and Eu-Jin Goh. Hierarchical identity based encryption with constant size ciphertext. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 440–456. Springer, Berlin, Heidelberg, May 2005. doi:10.1007/11426639_26.

BDD+24. Andrea Basso, Pierrick Dartois, Luca De Feo, Antonin Leroux, Luciano Maino, Giacomo Pope, Damien Robert, and Benjamin Wesolowski. SQIsign2D-West - the fast, the small, and the safer. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part III*, volume 15486 of *LNCS*, pages 339–370. Springer, Singapore, December 2024. doi:10.1007/978-981-96-0891-1_11.

BF23. Andrea Basso and Tako Boris Fouotsa. New SIDH countermeasures for a more efficient key exchange. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VIII*, volume 14445 of *LNCS*, pages 208–233. Springer, Singapore, December 2023. doi:10.1007/978-981-99-8742-9_7.

BGZ23. Dan Boneh, Jiaxin Guan, and Mark Zhandry. A lower bound on the length of signatures based on group actions and generic isogenies. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 507–531. Springer, Cham, April 2023. doi:10.1007/978-3-031-30589-4_18.

BKM+24. Benjamin Benčina, Péter Kutas, Simon-Philipp Merz, Christophe Petit, Miha Stopar, and Charlotte Weitkämper. Improved algorithms for finding fixed-degree isogenies between supersingular elliptic curves. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part V*, volume 14924 of *LNCS*, pages 183–217. Springer, Cham, August 2024. doi:10.1007/978-3-031-68388-6_8.

BM25. Andrea Basso and Luciano Maino. POKÉ: A compact and efficient PKE from higher-dimensional isogenies. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part II*, volume 15602 of *LNCS*, pages 94–123. Springer, Cham, May 2025. doi:10.1007/978-3-031-91124-8_4.

BMP23. Andrea Basso, Luciano Maino, and Giacomo Pope. FESTA: Fast encryption from supersingular torsion attacks. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VII*, volume 14444 of *LNCS*, pages 98–126. Springer, Singapore, December 2023. doi:10.1007/978-981-99-8739-9_4.

Boy08. Xavier Boyen. The uber-assumption family (invited talk). In Steven D. Galbraith and Kenneth G. Paterson, editors, *PAIRING 2008*, volume 5209 of *LNCS*, pages 39–56. Springer, Berlin, Heidelberg, September 2008. doi:10.1007/978-3-540-85538-5_3.

CD20. Wouter Castryck and Thomas Decru. CSIDH on the surface. In Jintai Ding and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020*, pages 111–129. Springer, Cham, 2020. doi:10.1007/978-3-030-44223-1_7.

CD23. Wouter Castryck and Thomas Decru. An efficient key recovery attack on SIDH. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 423–447. Springer, Cham, April 2023. doi:10.1007/978-3-031-30589-4_15.

CK20. Leonardo Colò and David Kohel. Orienting supersingular isogeny graphs. Cryptology ePrint Archive, Report 2020/985, 2020. URL: https://eprint.iacr.org/2020/985.

CLM+18. Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. CSIDH: An efficient post-quantum commutative group action. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part III*, volume 11274 of *LNCS*, pages 395–427. Springer, Cham, December 2018. doi:10.1007/978-3-030-03332-3_15.

DF24. Max Duparc and Tako Boris Fouotsa. SQIPrime: A dimension 2 variant of SQISignHD with non-smooth challenge isogenies. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part III*, volume 15486 of *LNCS*, pages 396–429. Springer, Singapore, December 2024. doi:10.1007/978-981-96-0891-1_13.

DFM20. Jelle Don, Serge Fehr, and Christian Majenz. The measure-and-reprogram technique 2.0: Multi-round fiat-shamir and more. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 602–631. Springer, Cham, August 2020. doi:10.1007/978-3-030-56877-1_21.

DFMS19. Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Security of the Fiat-Shamir transformation in the quantum random-oracle model. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 356–383. Springer, Cham, August 2019. doi:10.1007/978-3-030-26951-7_13.

DHK+23. Julien Duman, Dominik Hartmann, Eike Kiltz, Sabrina Kunzweiler, Jonas Lehmann, and Doreen Riepel. Generic models for group actions. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 406–435. Springer, Cham, May 2023. doi:10.1007/978-3-031-31368-4_15.

DKL+20. Luca De Feo, David Kohel, Antonin Leroux, Christophe Petit, and Benjamin Wesolowski. SQISign: Compact post-quantum signatures from quaternions and isogenies. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part I*, volume 12491 of *LNCS*, pages 64–93. Springer, Cham, December 2020. doi:10.1007/978-3-030-64837-4_3.

DLRW24. Pierrick Dartois, Antonin Leroux, Damien Robert, and Benjamin Wesolowski. SQIsignHD: New dimensions in cryptography. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 3–32. Springer, Cham, May 2024. doi:10.1007/978-3-031-58716-0_1.

FHL+25. Tako Boris Fouotsa, Marc Houben, Gioella Lorenzon, Ryan Rueger, and Parsa Tasbihgou. On the active security of the PEARL-SCALLOP group action, 2025.

FKL18. Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018. doi:10.1007/978-3-319-96881-0_2.

FMP23. Tako Boris Fouotsa, Tomoki Moriya, and Christophe Petit. M-SIDH and MD-SIDH: Countering SIDH attacks by masking information. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 282–309. Springer, Cham, April 2023. doi:10.1007/978-3-031-30589-4_10.

Fou25. Tako Boris Fouotsa. WaterSQI and PRISMO: Quaternion signatures for supersingular isogeny group actions. Cryptology ePrint Archive, Paper 2025/1737, 2025. URL: https://eprint.iacr.org/2025/1737.

GHHM21. Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Tight adaptive reprogramming in the QROM. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 637–667. Springer, Cham, December 2021. doi:10.1007/978-3-030-92062-3_22.

GLM24. Steven D. Galbraith, Yi-Fu Lai, and Hart Montgomery. A simpler and more efficient reduction of DLog to CDH for abelian group actions. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part II*, volume 14603 of *LNCS*, pages 36–60. Springer, Cham, April 2024. doi:10.1007/978-3-031-57725-3_2.

Hal05. Sean Hallgren. Fast quantum algorithms for computing the unit group and class group of a number field. In *STOC*, pages 468–474. ACM, 2005.

JM24. Joseph Jaeger and Deep Inder Mohan. Generic and algebraic computation models: When AGM proofs transfer to the GGM. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part V*, volume 14924 of *LNCS*, pages 14–45. Springer, Cham, August 2024. doi:10.1007/978-3-031-68388-6_2.

KHK25. Hyeonhak Kim, Seokhie Hong, and Suhri Kim. INKE: Fast isogeny-based PKE using intermediate curves. Cryptology ePrint Archive, Paper 2025/1458, 2025. URL: https://eprint.iacr.org/2025/1458.

Ler25. Antonin Leroux. Verifiable random function from the deuring correspondence and higher dimensional isogenies. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VII*, volume 15607 of *LNCS*, pages 167–194. Springer, Cham, May 2025. doi:10.1007/978-3-031-91098-2_7.

LZ19. Qipeng Liu and Mark Zhandry. Revisiting post-quantum Fiat-Shamir. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 326–355. Springer, Cham, August 2019. doi:10.1007/978-3-030-26951-7_12.

Mau05. Ueli M. Maurer. Abstract models of computation in cryptography (invited paper). In Nigel P. Smart, editor, *10th IMA International Conference on Cryptography and Coding*, volume 3796 of *LNCS*, pages 1–12. Springer, Berlin, Heidelberg, December 2005. doi:10.1007/11586821_1.

MMP+23. Luciano Maino, Chloe Martindale, Lorenz Panny, Giacomo Pope, and Benjamin Wesolowski. A direct key recovery attack on SIDH. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 448–471. Springer, Cham, April 2023. doi:10.1007/978-3-031-30589-4_16.

MMP25. Marzio Mula, Nadir Murru, and Federico Pintore. On random sampling of supersingular elliptic curves. *Annali di Matematica Pura ed Applicata (1923 -)*, 204(3):1293–1335, Jun 2025. doi:10.1007/s10231-024-01528-x.

MZ22. Hart Montgomery and Mark Zhandry. Full quantum equivalence of group action DLog and CDH, and more. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part I*, volume 13791 of *LNCS*, pages 3–32. Springer, Cham, December 2022. doi:10.1007/978-3-031-22963-3_1.

NO24. Kohei Nakagawa and Hiroshi Onuki. QFESTA: Efficient algorithms and parameters for FESTA using quaternion algebras. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part V*, volume 14924 of *LNCS*, pages 75–106. Springer, Cham, August 2024. doi:10.1007/978-3-031-68388-6_4.

NOC+24. Kohei Nakagawa, Hiroshi Onuki, Wouter Castryck, Mingjie Chen, Riccardo Invernizzi, Gioella Lorenzon, and Frederik Vercauteren. SQIsign2D-East: A new signature scheme using 2-dimensional isogenies. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part III*, volume 15486 of *LNCS*, pages 272–303. Springer, Singapore, December 2024. doi:10.1007/978-981-96-0891-1_9.

Onu21. Hiroshi Onuki. On oriented supersingular elliptic curves. *Finite Fields and Their Applications*, 69:101777, 2021.

OZ24. Emmanuela Orsini and Riccardo Zanotto. Simple two-message OT in the explicit isogeny model. *CiC*, 1(1):15, 2024. doi:10.62056/a39qgy4e-.

PR23. Aurel Page and Damien Robert. Introducing clapoti(s): Evaluating the isogeny class group action in polynomial time. Cryptology ePrint Archive, Report 2023/1766, 2023. URL: https://eprint.iacr.org/2023/1766.

PW24. Aurel Page and Benjamin Wesolowski. The supersingular endomorphism ring and one endomorphism problems are equivalent. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 388–417. Springer, Cham, May 2024. doi:10.1007/978-3-031-58751-1_14.

Rob22. Damien Robert. Evaluating isogenies in polylogarithmic time. Cryptology ePrint Archive, Report 2022/1068, 2022. URL: https://eprint.iacr.org/2022/1068.

Rob23. Damien Robert. Breaking SIDH in polynomial time. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 472–503. Springer, Cham, April 2023. doi:10.1007/978-3-031-30589-4_17.

Rob24. Damien Robert. On the efficient representation of isogenies (a survey). Cryptology ePrint Archive, Report 2024/1071, 2024. URL: https://eprint.iacr.org/2024/1071.

SEMR24. Maria Corte-Real Santos, Jonathan Komada Eriksen, Michael Meyer, and Krijn Reijnders. AprèsSQI: Extra fast verification for SQIsign using extension-field signing. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 63–93. Springer, Cham, May 2024. doi:10.1007/978-3-031-58716-0_3.

Sho97. Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EUROCRYPT'97*, volume 1233 of *LNCS*, pages 256–266. Springer, Berlin, Heidelberg, May 1997. doi:10.1007/3-540-69053-0_18.

Unr12. Dominique Unruh. Quantum proofs of knowledge. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 135–152. Springer, Berlin, Heidelberg, April 2012. doi:10.1007/978-3-642-29011-4_10.

Wat18. John Watrous. *The Theory of Quantum Information*. Cambridge University Press, USA, 1st edition, 2018.

Wes22. Benjamin Wesolowski. Orientations and the supersingular endomorphism ring problem. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 345–371. Springer, Cham, May / June 2022. doi:10.1007/978-3-031-07082-2_13.

Win99. A. Winter. Coding theorem and strong converse for quantum channels. *IEEE Transactions on Information Theory*, 45(7):2481–2485, 1999.

Zha22. Mark Zhandry. To label, or not to label (in generic groups). In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 66–96. Springer, Cham, August 2022. doi:10.1007/978-3-031-15982-4_3.

Zha25. Mark Zhandry. Quantum money from abelian group actions. *TheoretiCS, Volume 4, Article 18, 1-62*, 2025.

ZZK22. Cong Zhang, Hong-Sheng Zhou, and Jonathan Katz. An analysis of the algebraic group model. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part IV*, volume 13794 of *LNCS*, pages 310–322. Springer, Cham, December 2022. doi:10.1007/978-3-031-22972-5_11.

A Additional preliminaries

A.1 Cryptographic group actions

Definition A.1 (Group action). *Let G be a group with identity element e, and let $\mathcal{X}$ be a set. A map $\star : G \times \mathcal{X} \to \mathcal{X}$ is a group action if it has these properties:*

1. *Identity: $e \star x = x$ for all $x \in \mathcal{X}$.*
2. *Compatibility: $(g \cdot h) \star x = g \star (h \star x)$ for all $g, h \in G$ and $x \in \mathcal{X}$.*

A group action is called free if $g \star x = x$ for some $x \in \mathcal{X}$ implies that $g = e$. It is called transitive if for every $x, y \in \mathcal{X}$, there exists an element $g \in G$ such that $g \star x = y$.

To use group actions for cryptography, we need them to have additional properties.

Definition A.2 (Effective Group Action). *Let $(G, \mathcal{X}, \star)$ be a group action with the following properties:*

1. *G is finite and there exist efficient algorithms for membership testing, equality testing, (random) sampling, group operation and inversion.*
2. *The set $\mathcal{X}$ is finite and there exist efficient algorithms for membership testing and to compute a unique representation.*
3. *There exists a distinguished element $x_0 \in \mathcal{X}$ with known representation, which we call the origin.*
4. *The group action is free and transitive, and there exists an efficient algorithm to compute $g \star x$ given $g \in G$ and $x \in \mathcal{X}$.*

Then we say that $(G, \mathcal{X}, x_0, \star)$ is an effective group action (EGA).

To model the extra twist of isogeny-based group actions like CSIDH, the following variant was defined in [DHK+23].

Definition A.3 (Effective Group Action with Twists). *Let $(G, \mathcal{X}, x_0, \star)$ be an* EGA. *We say that it is an effective group action with twists* (EGAT)*, if there exists an efficient algorithm that, given $x = a \star x_0$, computes $x^t = -a \star x_0$.*

A.2 Mixed states

Density operators. When a system A is entangled with another system, or its preparation is not fully known, we use a density operator to describe the state of A. A density operator is a positive semi-definite matrix ρ of trace 1. Pure states $|\psi\rangle$ correspond to the special case $\rho = |\psi\rangle\langle\psi|$. If ρ_{AB} is the state of a system (A, B), then the state of A is obtained by tracing out B, i.e. $\rho_A = \mathrm{Tr}_B(\rho_{AB})$.

Trace distance. The trace norm of ρ is $\|\rho\|_1 = \mathrm{Tr}(\sqrt{\rho^\dagger \rho})$. It always holds that $\|\rho_A\|_1 \leq \|\rho_{AB}\|_1$. The trace distance of two density operators ρ and σ is $T(\rho, \sigma) = \frac{1}{2}\|\rho - \sigma\|_1$. The trace distance generalizes the statistical distance of random variables to quantum states. $T(\rho, \sigma)$ is the largest probability difference that ρ and σ could give to the same measurement outcome.

We will make use of the Gentle Measurement Lemma from [Win99, Wat18]. We present a restricted variant of the lemma that is sufficient for our purposes.

Lemma A.4. *Let ρ_{AB} be the joint state of a register A and a single-qubit register B. Let ε be an upper bound on the probability of measuring $B = 0$. Let $\rho_{A|1}$ be the post-measurement state of A conditioned on observing $B = 1$. Then*

$$T(\rho_A, \rho_{A|1}) \leq \sqrt{\varepsilon}.$$

B Further discussion

B.1 Why $\mathbf{E}_{\mathsf{end}}$ is needed – a separation result

In our AIM, we always give the adversary a curve $\mathbf{E}_{\mathsf{end}}$ with known endomorphism ring, in addition to the other curves it sees in the protocol. The question is whether it is necessary to always include $\mathbf{E}_{\mathsf{end}}$. Here, we will argue that if $\mathbf{E}_{\mathsf{end}}$ is not included, then our model will not be very useful. In particular, certain problems that are clearly easy in the standard model, can be shown to be "hard" in this variant of the AIM.

*Problem B.1 (*AnyEnd*).* Given a prime p and a curve $\mathbf{E} \leftarrow \mathsf{SS}(p)$ over $\mathbb{F}_{p^2}$, compute some supersingular curve $\mathbf{E}'$ over $\mathbb{F}_{p^2}$ and a non-scalar endomorphism of $\mathbf{E}'$ (in efficient representation).

This problem is similar to the OneEnd problem, where given $\mathbf{E}$, the goal is to compute some non-scalar endomorphism of $\mathbf{E}$. The difference is that in AnyEnd, the endomorphism can belong to any curve.

Lemma B.2. *In the standard model and the AIM with* $\mathbf{E}_{\mathsf{end}}$*,* AnyEnd *is solvable in polynomial time in* $\log p$*.*

Proof. Some curve $\mathbf{E}_{\mathsf{end}}$ of known endomorphism ring is always known. To solve AnyEnd, we ignore $\mathbf{E}$, and output $\mathbf{E}_{\mathsf{end}}$ and some non-scalar endomorphism of $\mathbf{E}_{\mathsf{end}}$. □

Lemma B.3. *In the AIM without* $\mathbf{E}_{\mathsf{end}}$*,* OneEnd *reduces to* AnyEnd*.*

Proof. Let $\mathcal{A}$ be a (quantum) polynomial time algebraic algorithm solving AnyEnd with probability $\varepsilon(p)$. We construct an algorithm $\mathcal{B}$ against OneEnd using $\mathcal{A}$. On input a uniformly random curve $\mathbf{E}$, we run $\mathcal{A}$ on $\mathbf{E}$. If $\mathcal{A}$ succeeds, it outputs $\mathbf{E}'$ and an endomorphism $\alpha : \mathbf{E}' \to \mathbf{E}'$. Furthermore, since $\mathcal{A}$ is algebraic, it outputs an isogeny $\sigma : \mathbf{E} \to \mathbf{E}'$. Then $\widehat{\sigma} \circ \alpha \circ \sigma$ is a non-scalar endomorphism of $\mathbf{E}$. Hence, $\mathcal{B}$ solves OneEnd with the same probability that $\mathcal{A}$ solves AnyEnd. □

B.2 On the existence of pathological games

By a pathological algebraic game, we mean games that are hard in the AIM, but easy in the standard model. Looking at the history of the AGM, this comes from different interpretations of what it means for a group element to "depend" on the input, which can lead to bugs, depending on the interpretation, e.g., [ZZK22]. Generally, there seems to be (at least) two types:

1. Games that exploit the type-system and/or syntax. The canonical example is given the bit-string $(X \| \perp)$, output the group element X. In the AGM, this is reducible to DLOG.
2. Cryptosystems with an if-then condition that when triggered trivially output the solution. It is only possible to trigger the if-then block in the standard model. This is the kind of contrived counter-example used to show that the ROM/GGM/AGM is not instantiable.

By restricting to type-safe games [Zha22], Zhandry could get rid of type 1 but not type 2. In the AGM of Jaeger and Mohan [JM24], both types are possible, but they suggest using informal interpretation to disallow these games. Looking at the AIM as we have defined it above, both types of pathological games are also possible and we require the same care when interpreting the model.

B.3 Oblivious sampling using the isogeny operation

It is possible to obliviously sample curves by computing isogeny in superposition (cf. for example [BBD+22, Section 7] and [Zha25, Theorem 5.3]). computing the isogeny operation in superposition, i Hence, quantum algebraic games, as they are defined now, could technically obliviously sample curves. In practice, we will never consider such games, so it does not really matter. One could indeed consider such a game another example of a pathological game.

If one really wanted to prevent this from being possible, we can define a new rule for the isogeny operation that fixes this. We can simply require that after a register with an isogeny φ has been used with isogeny, no unitary operation may ever operate on it again. This prevents the efficient representation of the isogeny from being uncomputed after the isogeny operation.

B.4 Extensions and open questions

Superpositions over curves as oracle output. We restrict to games where the adversary does not receive a superposition of curves. The issue is that we need to quantify what it means for an algorithm to have seen a curve. If a quantum-accessible oracle outputs a superposition of curves, then later measures the output to $\mathbf{E}_t$, our reduction has not seen the curve $\mathbf{E}_t$. Yet we

Game q-hint-EUF-NMA$_{\mathsf{SIG}[\Sigma]}(\mathcal{A})$

00 $(x, w) \leftarrow \mathsf{Gen}(1^\lambda)$
01 $h_1, \ldots, h_q \leftarrow \mathcal{H}_x$
02 $(\mathsf{msg}^\star, \mathsf{sig}^\star) \leftarrow \mathcal{A}^{\mathsf{RO}}(x, h_1, \ldots, h_q)$
03 **return** $\mathsf{Ver}_{\mathsf{SIG}[\Sigma]}(x, \mathsf{msg}^\star, \mathsf{sig}^\star)$

Fig. 11: The q-hint-EUF-NMA-game for a Fiat-Shamir signature scheme $\mathsf{SIG}[\Sigma]$ and hint distribution $\mathcal{H}$ in the QROM.

consider the curve $\mathbf{E}_t$ to be one of the curves that the adversary got as input, with index t. So when the algebraic adversary outputs an explanation (t, D), we cannot recover $\mathbf{E}_t$.

One workaround would be to not allow quantum oracles to output curves, as we do, which is sufficient for our purposes. There are however ways to capture the above scenario. For example, we could let the adversary access this concrete superposition via an index. This is how the AGAM [DHK+23] proves quantum-CCA security of group action ElGamal.

On explicitly capturing torsion points. It would be interesting to capture torsion points as concrete objects since there exist protocols that make more explicit use of them. For example, in the PKE scheme POKÉ [BM25], the shared key used to encrypt the message is a point and the underlying computational assumption requires to compute that point. However, formalizing this seems to require stronger assumptions or idealization.

On defining a generic isogeny model. We expect a generic model for isogenies to be much more complex. While intuitively, one could just provide handles to elliptic curves and an oracle for isogeny computations, it is not immediately clear how computations on points should be handled.

C Deferred proofs

C.1 Proof of Lemma 5.9

We split the proof of Lemma 5.9 into two reductions. As an intermediate step, we define the family of games q_s-hint-EUF-NMA in Fig. 11.

The first reduction is from hint-EUF-NMA to EUF-CMA. This reduction is almost identical to the EUF-NMA to EUF-CMA reduction in [GHHM21, Theorem 3]. The only changes we make are that:

1. We use hints for the simulation.
2. We use a worst-case definition for the commitment min-entropy.

With these modifications, their proof goes through as before.

Theorem C.1 (Adapted from [GHHM21, Theorem 3]). *Let $\mathcal{A}$ be a (quantum)* EUF-CMA *adversary issuing at most q_s classical queries to the signing oracle O_{SIGN} and at most q_r quantum queries to the random oracle* RO*. Then there exists a q_s-*hint-EUF-NMA *adversary $\mathcal{B}$ and a q_s-*hint-wHVZK *adversary $\mathcal{D}$ such that*

$$\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{SIG}[\Sigma]}(\mathcal{A}) \leq \mathsf{Adv}^{q_s\text{-}\mathsf{hint\text{-}EUF\text{-}NMA}}_{\mathsf{SIG}[\Sigma],\mathcal{H}}(\mathcal{B}) + \mathsf{Adv}^{q_s\text{-}\mathsf{hint\text{-}wHVZK}}_{\Sigma,\mathcal{H}}(\mathcal{D}) + \frac{3q_s}{2}\sqrt{(q_s + q_r + 1) \cdot \mathsf{MinEnt}(\Sigma)}$$

and the running time of $\mathcal{B}$ and $\mathcal{D}$ is about that of $\mathcal{A}$.

The second reduction is from hint-ID-Sound to hint-EUF-NMA. This reduction relies on the measure-and-reprogram technique from [DFMS19, DFM20].

Theorem C.2 ([DFM20, Theorem 2]). *Let $\mathcal{X}$ and $\mathcal{Y}$ be finite non-empty sets. There exists a black-box two-stage quantum algorithm $\mathcal{T}$ with the following property. Let $\mathcal{A}$ be a quantum oracle algorithm issuing at most q_r quantum queries to a uniformly random $\mathsf{RO} : \mathcal{X} \to \mathcal{Y}$ and that outputs some classical $x \in \mathcal{X}$ and a (possibly quantum) output z. Then, the two-stage algorithm $\mathcal{T}^{\mathcal{A}}$ outputs some $x \in \mathcal{X}$ in the first stage and, upon a random $y \in \mathcal{Y}$ as input to the second stage, a (possibly quantum) output z, so that for any $x_0 \in \mathcal{X}$ any (possible quantum) predicate V:*

$$\Pr_y\left[x = x_0 \wedge V(x, y, z) = 1 \;\middle|\; (x, z) \leftarrow \langle \mathcal{T}^{\mathcal{A}}, y\rangle\right] \geq \frac{1}{(2q_r+1)^2} \cdot \Pr_{\mathsf{RO}}\left[x = x_0 \wedge V(x, \mathsf{RO}(x), z) = 1 \;\middle|\; (x, z) \leftarrow \mathcal{A}^{\mathsf{RO}}\right].$$

Furthermore, $\mathsf{Time}(\mathcal{T}^{\mathcal{A}}) = \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\log|\mathcal{X}|, \log|\mathcal{Y}|, q_r)$.

In the first stage, the algorithm $\mathcal{T}^{\mathcal{A}}$ stops $\mathcal{A}$ right after it makes one of its q_r quantum queries. It measures the query x and outputs it. In the second stage, $\mathcal{T}^{\mathcal{A}}$ is given a random y, and then reprograms the random oracle so that $\mathsf{RO}(x) := y$. After that, it keeps running $\mathcal{A}$ until it terminates with output (x', z). $\mathcal{T}^{\mathcal{A}}$ wins if $x = x'$ and the predicate V is satisfied.

The measure-and-reprogram technique can be applied straightforwardly to obtain a reduction from hint-ID-Sound to hint-EUF-NMA. We recall that for Fiat-Shamir signatures, we have

$$\mathsf{RO} : \{0,1\}^{\leq u} \to \mathcal{C}.$$

Lemma C.3. *Let $\mathcal{A}$ be a (quantum) adversary for q_s-hint-EUF-NMA issuing at most q_r quantum queries to the random oracle RO. Then there exists a (quantum) adversary $\mathcal{B}$ for q_s-hint-ID-Sound such that*

$$\mathsf{Adv}^{q_s\text{-hint-EUF-NMA}}_{\mathsf{SIG}[\Sigma],\mathcal{H}}(\mathcal{A}) \leq (2q_r+1)^2 \cdot \mathsf{Adv}^{q_s\text{-hint-ID-Sound}}_{\Sigma,\mathcal{H}}(\mathcal{B})$$

and the running time of $\mathcal{B}$ is about that of $\mathcal{A}$.

Proof. We can modify $\mathcal{A}$ such that its output is of the form $(\mathsf{query}, \mathsf{rsp})$ with $\mathsf{query} = (x, \mathsf{com}, \mathsf{msg})$. We define

$$V(\mathsf{query}, \mathsf{chl}, \mathsf{rsp}) := \mathsf{Ver}_{\Sigma}(x, \mathsf{com}, \mathsf{chl}, \mathsf{rsp}).$$

$\mathcal{A}$ wins the hint-EUF-NMA game if $V(\mathsf{query}, \mathsf{RO}(\mathsf{query}), \mathsf{rsp}) = 1$.

We define the adversary $\mathcal{B}$ for q_s-hint-ID-Sound using the two-stage algorithm $\mathcal{T}^{\mathcal{A}}$ from Theorem C.2. $\mathcal{B}$ runs the first stage of $\mathcal{T}^{\mathcal{A}}$ to obtain $\mathsf{query} = (x, \mathsf{com}, \mathsf{msg})$. Next, it calls $\mathsf{QO}_{\mathrm{SWAP}}(\mathsf{com})$ to send com to the game. Then, it obtains $\mathsf{chl} \leftarrow \mathsf{O}_{\mathrm{CHAL}}$ and runs the second stage of $\mathcal{T}^{\mathcal{A}}$ with chl. Finally, $\mathcal{T}^{\mathcal{A}}$ outputs $(\mathsf{query}', \mathsf{rsp})$. $\mathcal{B}$ wins if $\mathsf{query} = \mathsf{query}'$ and $V(\mathsf{query}, \mathsf{chl}, \mathsf{rsp}) = 1$. We conclude using Theorem C.2. □

By combining Theorem C.1 and Lemma C.3, we obtain the result in Lemma 5.9.

C.2 Proof of Theorem 5.11

Proof. Let $\mathcal{A}$ be a (quantum) algebraic adversary for the EUF-CMA-game of SQIsign. By lifting Lemma 5.9 to the AIM, we obtain two algebraic algorithms $\mathcal{M}$ and $\mathcal{N}$ such that

$$\begin{aligned}\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{SIG}[\Sigma]}(\mathcal{A}) \leq{}& (2q_r+1)^2 \cdot \mathsf{Adv}^{q_s\text{-hint-ID-Sound}}(\mathcal{M}) \\ &+ \mathsf{Adv}^{q_s\text{-hint-wHVZK}}(\mathcal{N}) \\ &+ \frac{3q_s}{2}\sqrt{(q_s+q_r+1)\cdot \mathsf{MinEnt}(\Sigma_{\mathsf{SQI}})}.\end{aligned} \tag{1}$$

By Theorem 5.10, there is an algebraic adversary $\mathcal{E}_1$ for q_s-hint-OneEnd such that

$$\mathsf{Adv}^{q_s\text{-hint-ID-Sound}}(\mathcal{M}) \leq \mathsf{Adv}^{q_s\text{-hint-OneEnd}}(\mathcal{E}_1) + \frac{1}{2^{e_{\mathsf{chl}}}} + \frac{1}{2\sqrt{p}}. \tag{2}$$

By Lemma 5.6, there exist two algorithms $\mathcal{D}$ and $\mathcal{F}$ such that

$$\mathsf{Adv}^{q_s\text{-hint-wHVZK}}(\mathcal{N}) \leq \mathsf{Adv}^{q_s\text{-hint-dist}}(\mathcal{D}) + \mathsf{Adv}^{\mathsf{rel}}_{R_{\mathsf{SQI}}}(\mathcal{F}) + \frac{q_s}{\sqrt{p}}. \tag{3}$$

Since $\mathcal{D}$ only outputs a single bit in the hint-dist game, it is trivially algebraic. Inspecting the proof of Lemma 5.6 (in particular [ABD+25, Lemma 4.5]), $\mathcal{B}$ is classical and does not obliviously sample curves, so it can be easily converted to an algebraic algorithm.

We also have $\mathsf{MinEnt}(\Sigma_{\mathsf{SQI}}) \leq \sqrt{p}$ by Lemma 5.6. Plugging this bound, (2), (3) into (1), we get

$$\begin{aligned}\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{SIG}[\Sigma]}(\mathcal{A}) \leq{}& (2q_r+1)^2 \cdot \mathsf{Adv}^{q_s\text{-hint-OneEnd}}(\mathcal{E}_1) \\ &+ \mathsf{Adv}^{q_s\text{-hint-dist}}(\mathcal{D}) + \mathsf{Adv}^{\mathsf{rel}}_{R_{\mathsf{SQI}}}(\mathcal{F}) \\ &+ \frac{(2q_r+1)^2}{2^{e_{\mathsf{chl}}}} + \frac{(2q_r+1)^2+2q_s}{2\sqrt{p}} + \frac{3q_s\sqrt{q_s+q_r+1}}{2\sqrt[4]{p}}\end{aligned} \tag{4}$$

Next, we use $\mathcal{F}$ to construct an algorithm $\mathcal{E}_2$ for q_s-hint-OneEnd problem. Given a curve $\mathbf{E}$ and some hints, $\mathcal{E}_2$ ignores the hints, and runs $\mathcal{F}$ to compute the witness, the quaternion ideal I. If $\mathcal{F}$ succeeds, we can recover $\mathrm{End}(\mathbf{E})$ from I. The curves output by $\mathsf{Gen}_{R_{\mathsf{SQI}}}$ have statistical distance at most $\frac{1}{2}p^{-1/2}$ from the stationary distribution by Lemma 5.7. Hence,

$$\mathsf{Adv}^{\mathsf{rel}}(\mathcal{F}) \leq \mathsf{Adv}^{q_s\text{-hint-OneEnd}}(\mathcal{E}_2) + \frac{1}{2\sqrt{p}} \tag{5}$$

Out of $\mathcal{E}_1$ and $\mathcal{E}_2$, let $\mathcal{B}$ be the algorithm that has the largest advantage for the q_s-hint-OneEnd problem. Using $(2q_r+1)^2+1 < 4(q_r+1)^2$, we obtain that

$$\begin{aligned}\mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathsf{SIG}[\Sigma]}(\mathcal{A}) \leq{}& 4(q_r+1)^2 \cdot \mathsf{Adv}^{q_s\text{-hint-OneEnd}}(\mathcal{B}) + \mathsf{Adv}^{q_s\text{-hint-dist}}(\mathcal{D}) \\ &+ \frac{4(q_r+1)^2}{2^{e_{\mathsf{chl}}}} + \frac{2(q_r+1)^2+q_s}{\sqrt{p}} + \frac{2q_s\sqrt{q_s+q_r+1}}{\sqrt[4]{p}}\end{aligned} \tag{6}$$

The last conditional reduction from q_s-hint-EndRing to q_s-hint-OneEnd is from [ABD+25, Theorem 6]. □

D AGAM to the AIM for quantum games

In this section, we strengthen Theorem 4.6 to apply for quantum games. We make use of the quantum computing preliminaries from Appendix A.2.

Theorem D.1. *Let* G *be an algebraic game in the* O-AGAM*. Let* $\mathcal{A}$ *be an algebraic algorithm for* G *in the* AIM *(resp.* O-AGAM*). Let* n *be an upper bound on the number of curves output by* $\mathcal{A}$*, and* d *an upper bound on the degree (resp. norm) of their explanations. Finally, let* $t \in \mathbb{N}$*. Then there is an algebraic algorithm* $\mathcal{B}$ *for* G *in the* O-AGAM *(resp.* AIM*), with*

$$\begin{aligned}\mathsf{Adv}^{\mathsf{G}}(\mathcal{B}) &\geq \mathsf{Adv}^{\mathsf{G}}(\mathcal{A}) - 2n\sqrt{2^{-t}}, \\ \mathsf{Time}^{\mathsf{G}}(\mathcal{B}) &\leq \mathsf{Time}^{\mathsf{G}}(\mathcal{A}) + nt \cdot \mathsf{poly}(\log p, \log d, \log|\mathrm{disc}\,\mathfrak{O}|).\end{aligned}$$

If $\mathcal{A}$ *is* AIM*-algebraic or a quantum algorithm, then* $\mathcal{B}$ *is a quantum algorithm.*

Proof. The idea is to convert the explanations $\mathcal{A}$ outputs using the previous algorithms. If $\mathcal{A}$ is O-AGAM-algebraic, we use Lemma 4.1, and if $\mathcal{A}$ is AIM-algebraic, we use Lemma 4.5.

In both cases, we have a conversion algorithm C for classical explanations. C expects the following inputs (as bits):

- The oriented curves $(E_0,\iota_0),(E_1,\iota_1),\ldots,(E_m,\iota_m)$ that $\mathcal{A}$ got as input.
- An oriented curve (E,ι) that $\mathcal{A}$ has output.

– A valid explanation for (E, ι).

If the input is not valid, we define the output of C to be 0.

However, C is not a PPT algorithm, it runs in expected time $\mathbb{E}[T] = \mathsf{poly}(\log p, \log d, \log|\operatorname{disc}(\mathfrak{O})|)$. We convert C to a PPT algorithm D as follows. By Markov's inequality, $\Pr\left[T > 2 \cdot \mathbb{E}[T]\right] \leq 1/2$. Let C' be the algorithm C modified to abort after time $2 \cdot \mathbb{E}[T]$. Let D be the algorithm that runs C' at most t times, each time with new randomness. Then D aborts with probability $\leq 2^{-t}$. Observe that in our case, if D does not fail, its output is uniquely determined by the input.

By a standard purification argument, we can implement D using a unitary U, incurring a polynomial blow-up in the circuit size. U operates on four registers: The input register $\mathtt{Q}.I$, the output register $\mathtt{Q}.O$, the success bit $\mathtt{Q}.B$ and the temporary working register $\mathtt{Q}.W$. On a classical input x, U computes

$$U : |x\rangle_I |0\rangle_B |0\rangle_O |0\rangle_W \mapsto \begin{array}{l} \sqrt{1-\varepsilon(x)}\ |x\rangle_I |1\rangle_B |D(x)\rangle_O |W_x^{(1)}\rangle_W \\ +\ \sqrt{\varepsilon(x)}\ |x\rangle_I |0\rangle_B |0\rangle_O |W_x^{(0)}\rangle_W \end{array},$$

where $|W_x^{(0)}\rangle$ and $|W_x^{(1)}\rangle$ might be in superposition and $\varepsilon(x) \leq 2^{-t}$ is the failure probability on input x. Afterwards, we measure $\mathtt{Q}.B$. If $\mathtt{Q}.B = 1$, the state of $\mathtt{Q}.O$ collapses to the uniquely determined output $D(x)$.

Let us now define the algorithm $\mathcal{B}$. Internally, it runs $\mathcal{A}$. When $\mathcal{A}$ outputs a (quantum) curve $(\mathtt{Q}.\mathbf{E}, \mathtt{Q}.\iota)$ and explanation $\mathtt{Q}.X$, $\mathcal{B}$ runs U (in superposition), to compute the new explanation. For simplicity, $\mathcal{B}$ uses a new set of registers $\mathtt{Q}.I_i, \mathtt{Q}.O_i, \mathtt{Q}.B_i, \mathtt{Q}.W_i$ every time it runs U. After running U, $\mathcal{B}$ measures $\mathtt{Q}.B_i$. If $\mathtt{Q}.B_i = 0$, then $\mathcal{B}$ aborts. Else, $\mathcal{B}$ outputs the curve and the new explanation.

We bound the advantage of $\mathcal{B}$ using a hybrid argument. We define the algorithms $H_0, \ldots, H_n$, where H_i converts the first i explanations. We have $H_0 = \mathcal{A}$ and $H_n = \mathcal{B}$. For $i \in \{1, \ldots, n\}$, the task at hand is to relate the advantage of H_{i-1} and H_i.

Let Z be the closed system of the game, i.e. the complete set of registers used by $\mathcal{A}$, the hybrid algorithms and the game. After the i-th output of H_{i-1} or H_i, the rest of what happens in the game constitutes a measurement on Z, to determine if the adversary won the game. This measurement can no longer depend on the registers $\mathtt{Q}.X_i, \mathtt{Q}.I_i, \mathtt{Q}.O_i, \mathtt{Q}.B_i, \mathtt{Q}.W_i$. Firstly, the game does not depend on the explanations. Secondly, after outputting $\mathtt{Q}.X_i$, $\mathcal{A}$ can no longer perform operations on it. Lastly, H_i will never again use $\mathtt{Q}.I_i, \mathtt{Q}.O_i, \mathtt{Q}.B_i, \mathtt{Q}.W_i$. In conclusion, after the i-th output of H_{i-1} or H_i, these registers do not affect the game outcome. Let Z_i be the remaining registers of Z.

H_{i-1} is identical to H_i up to the point when $\mathcal{A}$ outputs its i-th curve. Let ρ be the state of Z_i at this point. When H_i computes U, this does not alter any registers in Z_i, so the state of Z_i is unaffected. Finally, H_i measures $\mathtt{Q}.B_i$ and aborts if $\mathtt{Q}.B_i = 0$. This happens with probability $\leq 2^{-t}$. If $\mathtt{Q}.B_i = 1$, let $\rho_{|1}$ be the resulting state of Z_i. By Lemma A.4, $T(\rho, \rho_{|1}) \leq \sqrt{2^{-t}}$.

These observations allow us to bound the advantage of H_i. Let M_i be the same algorithm as H_i, except that it does not abort if $\mathtt{Q}.B_i = 0$.

$$\begin{aligned} \Pr\left[\mathsf{G}(H_i) = 1\right] &= \Pr\left[\mathtt{Q}.B_i = 1\right] \cdot \Pr\left[\mathsf{G}(M_i) = 1 \;\middle|\; \mathtt{Q}.B_i = 1\right] \\ &\geq \Pr\left[\mathsf{G}(M_i) = 1 \;\middle|\; \mathtt{Q}.B_i = 1\right] - \Pr\left[\mathtt{Q}.B_i = 0\right] \\ &\geq \Pr\left[\mathsf{G}(H_{i-1}) = 1\right] - T(\rho, \rho_{|1}) - \Pr\left[\mathtt{Q}.B_i = 0\right] \\ &\geq \Pr\left[\mathsf{G}(H_{i-1}) = 1\right] - 2\sqrt{2^{-t}} \end{aligned}$$

By induction, we conclude that $\mathsf{Adv}^{\mathsf{G}}(\mathcal{B}) \geq \mathsf{Adv}^{\mathsf{G}}(\mathcal{A}) - 2n\sqrt{2^{-t}}$. □

E The DLOG-to-CDH reduction with a fixed-basis CDH oracle

In this section, we sketch how the DLOG-to-CDH reduction changes when the CDH oracle no longer uses a variable starting curve, but it is instead limited to a fixed curve of known endomorphism ring.

E.1 Problem statements

Problem E.1 (CDH for generalized SIDH key exchanges). Let $\mathbf{E}_0$ be a fixed supersingular elliptic curve defined over $\mathbb{F}_{p^2}$ of known endomorphism ring. Let $\mathfrak{A}, \mathfrak{B}$ be lists of subgroups of $\mathbf{E}_0(\mathbb{F}_{p^2})$. Let $\varphi_A : \mathbf{E}_0 \to \mathbf{E}_A$ and $\varphi_B : \mathbf{E}_0 \to \mathbf{E}_B$ be an $\mathfrak{A}$-isogeny and a $\mathfrak{B}$-isogeny, respectively. Given $(\mathbf{E}_A, \varphi_A(\mathfrak{B}))$ and $(\mathbf{E}_B, \varphi_B(\mathfrak{A}))$, compute the curve $\mathbf{E}_{AB}$ that is the codomain of the isogeny $[\varphi_A]_*(\varphi_B) = [\varphi_B]_*(\varphi_A)$.

In this case, since the basis is fixed, we no longer need to define a conjoined DLOG problem.

Problem E.2 (DLOG for generalized SIDH key exchanges). Let $\mathbf{E}_0$ be a fixed random supersingular elliptic curve defined over $\mathbb{F}_{p^2}$ with a list $\mathfrak{A}$ of subgroups of $\mathbf{E}_0(\mathbb{F}_{p^2})$. Let $\varphi_A : \mathbf{E}_0 \to \mathbf{E}_A$ be an $\mathfrak{A}$-isogeny. Given the $(\mathbf{E}_A, \varphi_A(\mathfrak{B}))$, compute an isogeny $\varphi_A^* : \mathbf{E}_0 \to \mathbf{E}_A$ such that $\varphi_A^*(\mathfrak{B}) = \varphi_A(\mathfrak{B})$.

Problem E.3 (weakDLOG for generalized SIDH key exchanges). Let $\mathbf{E}_0$ be a fixed random supersingular elliptic curve defined over $\mathbb{F}_{p^2}$ with a list $\mathfrak{A}$ of subgroups of $\mathbf{E}_0(\mathbb{F}_{p^2})$. Let $\varphi_A : \mathbf{E}_0 \to \mathbf{E}_A$ be an $\mathfrak{A}$-isogeny. Given the $(\mathbf{E}_A, \varphi_A(\mathfrak{B}))$, compute an isogeny $\varphi_A^* : \mathbf{E}_0 \to \mathbf{E}_A$ such that $\varphi_A^*(\mathfrak{B}) = \varphi_A(\mathfrak{B})$, or a non-trivial endomorphism in $\mathrm{End}(\mathbf{E}_A)$.

E.2 The reduction

The first step of the reduction is similar to the case with variable starting curves.

Theorem E.4. *For any algebraic PPT adversary $\mathcal{A}$ that solves the CDH problem for generalized SIDH key exchanges with a fixed basis curve $\mathbf{E}_0$, there exists a PPT algorithm $\mathcal{B}$ such that*

$$\mathsf{Adv}^{\mathsf{CDH}}(\mathcal{A}) \leq 10 \cdot \mathsf{Adv}^{\mathsf{weakDLOG}}(\mathcal{B}).$$

Proof (sketch). The proof proceeds similarly as the proof of Theorem 6.7. In the fixed-basis case, an adversary (and our reduction) may come equipped with a basis for $\mathrm{End}(\mathbf{E}_0)$ baked in. Hence, it makes sense to treat $\mathbf{E}_0$ as a curve of known endomorphism ring, and set $\mathbf{E}_{\mathsf{end}} = \mathbf{E}_0$. This simplifies the reduction, as the case $\mathsf{expl} = 0$ is no longer needed.

Since we no longer have the case $\mathsf{expl} = 0$, the algorithm $\mathcal{B}$ has a higher chance (1/5) of guessing the correct case. However, the weakDLOG problem (Problem E.3) is now only with respect to a single instance, so the reduction has to get two weakDLOG challenges (one with respect to $\mathfrak{A}$ and one with respect to $\mathfrak{B}$), which explains the loss factor 10 in the theorem. □

The second step of the reduction, i.e., the equivalent of Theorem 6.8 in the fixed-basis setting, is not possible: given a uniformly random curve, it is hard to embed it into a CDH-with-fixed-basis challenge, where the curves are at a prescribed distance from the starting curve. This means that if we only have access to a fixed-basis CDH oracle, the DLOG problem and the CDH problem are equivalent only assuming the hardness of the OneEnd problem for the key exchange public key curves.